# Kubernetes Engineering Mastery — Expanded Teaching Edition

## From Linux Beginner to Senior / Staff Kubernetes & Platform Engineer

**Edition:** 26 October 2026 | **Course duration target:** 36 weeks plus continued portfolio work | **Original core course:** retained in full | **New material:** guided deep lessons with diagram-arrow explanations, command reasoning, troubleshooting and field-level YAML commentary.

**Career target:** ₹1 crore annual total compensation is an aspiration, not a course guarantee; it typically requires high-impact engineering experience, interviewing, business impact, and sometimes equity. **Technical baseline:** Kubernetes v1.37 series as listed by the official release page on 26 October 2026. Always verify your actual cluster minor/patch version. **Lab baseline:** Debian 12, 8 GB RAM, single-node kind cluster wherever possible. Never run privileged changes, paid cloud provisioning or disruptive commands on shared systems without authorization.

**How to use this edition:** First read Deep Lessons 01–20 in order. Every figure names its nodes and explains each directional arrow. Read each command's `Where / Why / Decode / Expected / Failure / Safety` before typing it. When a manifest is shown, explain it out loud using the fields list. The original 36-week course is preserved after the deep lessons and remains your weekly assessment plan. The `reference/EXPLAINED-ORIGINAL-MANIFESTS.md` companion annotates each original guided-lab YAML file.

**Verification truth:** Static code, schema and PDF rendering checks are not live-cluster validation. The commands and expected outputs are teaching examples unless evidence of actual execution is supplied. Several advanced labs (multi-node HA, real CSI, AWS EKS, full controller runtime) demand a larger authorized environment; do not claim these as completed until executed and tested.

# Deep Lesson 01 — The terminal, Linux processes and YAML grammar

**Goal:** Understand the shell, process inspection, files, quoting, pipes, and YAML indentation before touching cluster state.
**Starting point:** No Kubernetes knowledge is required; use an ordinary Debian terminal.
**Hinglish mental model:** Linux mein command ek program ya shell built-in hota hai; `kubectl` ka output bhi text hai, magic nahi.

## Teach the concept from zero

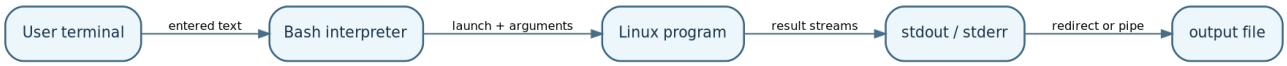
A terminal is a text interface to a shell (typically Bash). The shell reads your command, expands variables and paths, and starts programs. A **process** is an executing program; it has a PID, permissions, open files and environment variables. Kubernetes ultimately runs Linux processes, so basic inspection matters. `pwd` answers where you are; `ls` lists entries; `cd` changes directories; `cat` displays file bytes. A pipeline with `|` sends the output of one process to the input of another. `>` creates or replaces a file; `>>` appends. Be careful: redirection happens before a command finishes and can erase an existing file.

A command generally looks like `program subcommand resource-name --flag=value -n namespace`. The shell interprets spaces unless a value is quoted. `kubectl` then parses its own arguments. For example, `-n artha-lab` selects a namespace; it does **not** create one. `-o yaml` selects the format of a response; it does not change objects. `--` separates command-line tool arguments from a command to run inside a container.

**YAML** is not a programming language. It is a structured way to describe data. A key followed by a colon introduces a value or nested map; `-` introduces a list item. `metadata.name` identifies an object; `spec` describes desired behavior; `status` is usually observed behavior written by Kubernetes. The same label (`app: web`) can appear in multiple objects because Services and controllers use selectors to find matching Pods. Strings that look like booleans can be different from real booleans: `"false"` is text; `false` is a boolean. Quoting is often safest for ConfigMap values and resource quantities. Never place production passwords in training files.

The **rule for safe Kubernetes practice** is: check which context and namespace you are operating on, inspect the current state, make one deliberate change, inspect events and outputs, then record cleanup steps. A command copied from a tutorial can mutate production if your kubeconfig points at production.

**Figure 1: The terminal, Linux processes and YAML grammar**




**How to read this diagram, arrow by arrow:**

- **User terminal → Bash interpreter** (entered text): The shell reads a line and expands quotes, variables and path patterns.
- **Bash interpreter → Linux program** (launch + arguments): The shell launches an executable (or runs a shell built-in) with parsed arguments.
- **Linux program → stdout / stderr** (result streams): The program writes useful data to standard output and errors to standard error.
- **stdout / stderr → output file** (redirect or pipe): Redirection or a pipe stores/transforms output; `>` truncates files, while `>>` appends.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 1: `pwd`

**Where:** HOST
**Why run it:** Confirm the directory in which you will create lab files.
**Decode every important part:** `pwd` means print working directory.
**Expected observation:** An absolute path such as `/home/user/k8s-labs`.
**If the result differs:** If the path is unexpected, use `cd` to move intentionally.
**Safety / side effect:** Read only.

```
pwd
```

### Command 2: `mkdir -p "$HOME/k8s-labs"`

**Where:** HOST
**Why run it:** Create a safe working folder.
**Decode every important part:** `mkdir` makes directories; `-p` creates missing parents and is not an error if the directory already exists; `$HOME` is a shell variable.
**Expected observation:** A directory is present; a successful run is normally silent.
**If the result differs:** Check permissions with `ls -ld "$HOME"`.
**Safety / side effect:** Creates a local directory, not a cluster resource.

```
mkdir -p "$HOME/k8s-labs"
```

### Command 3: `ls -lah "$HOME/k8s-labs"`

**Where:** HOST
**Why run it:** Inspect files and ownership.
**Decode every important part:** `ls` lists; `-l` detailed, `-a` hidden files, `-h` human-readable sizes.
**Expected observation:** A detailed listing, or `total 0` when empty.
**If the result differs:** Use `pwd` and confirm you typed the right path.
**Safety / side effect:** Read only.

```
ls -lah "$HOME/k8s-labs"
```

### Command 4: `cat /etc/os-release`

**Where:** HOST
**Why run it:** Identify your Linux distribution without assumptions.
**Decode every important part:** `cat` prints the named file; `/etc/os-release` contains distro metadata.
**Expected observation:** Fields such as `ID=debian`.
**If the result differs:** If unavailable, inspect `uname -a` and the environment.
**Safety / side effect:** Read only.

```
cat /etc/os-release
```

### Command 5: `free -h`

**Where:** HOST
**Why run it:** Check realistic lab RAM.
**Decode every important part:** `free` displays memory; `-h` changes units to human-readable values.
**Expected observation:** Total/used/available columns; available matters more than free.
**If the result differs:** Close memory-heavy programs before making the cluster.
**Safety / side effect:** Read only.

```
free -h
```

### Command 6: `printf "hello\n"`

**Where:** HOST
**Why run it:** See how escape characters work.
**Decode every important part:** `printf` formats output; `\n` is a newline escape in the format string.
**Expected observation:** One line containing `hello`.
**If the result differs:** Quote the format string to prevent unintended shell interpretation.
**Safety / side effect:** Read only.

```
printf "hello\n"
```

## Common faults and diagnostic reasoning

- **Permission denied** — The user may lack filesystem write permission; do not use `sudo` blindly.
- **Unexpected file overwrite** — Before `>` or deleting paths, inspect the destination and prefer a new study folder.
- **YAML validation error** — Count indentation in spaces; `kubectl explain` reveals API fields but does not fix syntax.

## Guided practice and evidence

1. Create `notes.md` in `~/k8s-labs`, record Linux version and available memory.
2. Write a five-line YAML object and identify map, list, scalar and indentation.
3. Explain what the shell handles versus what kubectl handles in a full command.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** What is the difference between stdout, a YAML `spec`, and the Kubernetes API?

**Model answer:** stdout is an output byte stream of a Linux process. YAML is a representation of data on disk or stdin. The API server accepts objects described by that data; an object spec declares desired behavior and is not itself an executing process.

**Official references:** https://www.gnu.org/software/bash/manual/bash.html, https://kubernetes.io/docs/concepts/overview/working-with-objects/

---

# Deep Lesson 02 — Containers, images, namespaces, and cgroups

**Goal:** Explain why a Linux container is isolated but shares a kernel, and what a Pod actually groups.
**Starting point:** You can run `ps`, `uname`, and view Linux `/proc`.
**Hinglish mental model:** Container ek isolated process group hai, mini virtual machine nahi.

## Teach the concept from zero

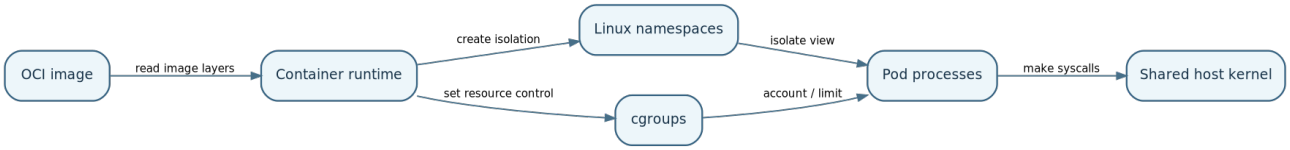
A **container image** is a read-only specification of filesystem layers and metadata. A container runtime combines those layers with a writable layer, namespaces, cgroups, and process startup. An image does not run by itself; a container is an instantiated process environment. Linux **namespaces** constrain what a process can see (such as PID trees, mounts, networks, hostname and user IDs). **cgroups** group processes for resource accounting and limits (CPU, memory, I/O). An unprivileged container does not get a separate kernel like a full VM; it shares the host kernel and must be hardened against kernel/privilege risks.

A Kubernetes **Pod** is the smallest schedulable compute unit. Containers in a Pod generally share the Pod network namespace and IP address, communicate over `localhost`, and may share declared volumes. They need not share the same image or process namespace. A sidecar is a cooperating container; an init container runs prior to the main workload. The Pod is assigned to one node. Individual containers may restart; a terminated Pod in a Deployment is replaced through controllers with a different Pod identity.

CPU requests primarily influence scheduler capacity accounting; limits can enforce CPU throttling through cgroups. Memory limits can result in an OOM kill. Do not confuse image size, image download behavior, memory reservation and runtime resident memory. On an 8 GB laptop, minimal resource requests and no giant observability stack are key.

**Architecture implication:** privileged container access can approach host-level power, and giving workloads unrestricted Linux capabilities defeats many isolation goals. The right mental model is layers of isolation, not a perfect sandbox.

**Figure 2: Containers, images, namespaces, and cgroups**




**How to read this diagram, arrow by arrow:**

- **OCI image → Container runtime** (read image layers): The runtime unpacks or mounts layers and chooses a process entrypoint.
- **Container runtime → Linux namespaces** (create isolation): PID/mount/network namespaces determine what processes can see.
- **Container runtime → cgroups** (set resource control): The runtime configures cgroups so resources can be accounted for or limited.
- **Linux namespaces → Pod processes** (isolate view): Processes see namespaced resources while still being host processes.
- **cgroups → Pod processes** (account / limit): Cgroup settings constrain or meter CPU, RAM and other resources.
- **Pod processes → Shared host kernel** (make syscalls): Containerized applications invoke the host Linux kernel; there is not a separate guest kernel.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 7: `uname -r`

**Where:** HOST
**Why run it:** Identify the shared Linux kernel version.
**Decode every important part:** `uname` reports system information; `-r` means kernel release.
**Expected observation:** A version string; containers on this Linux host normally share that kernel.
**If the result differs:** A VM/container host may report a kernel different from your OS userland.
**Safety / side effect:** Read only.

```
uname -r
```

### Command 8: `ps -eo pid,ppid,comm --sort=pid | head`

**Where:** HOST
**Why run it:** Learn PID, parent PID and process name.
**Decode every important part:** `ps -e` selects processes; `-o` chooses columns; `--sort=pid` orders; `| head` limits rows.
**Expected observation:** A table of processes.
**If the result differs:** If flags vary, consult `man ps`; inside containers you may see fewer PIDs.
**Safety / side effect:** Read only.

```
ps -eo pid,ppid,comm --sort=pid | head
```

### Command 9: `cat /proc/self/cgroup`

**Where:** HOST
**Why run it:** Inspect your own process cgroup membership.
**Decode every important part:** `/proc` exposes kernel process information; `self` means the reading process.
**Expected observation:** A hierarchy path such as `0::/user.slice/...` on cgroup v2.
**If the result differs:** The layout changes by host and init system.
**Safety / side effect:** Read only.

```
cat /proc/self/cgroup
```

### Command 10: `docker ps --format "table {{.Names}}\t{{.Status}}"`

**Where:** HOST; Docker installed
**Why run it:** List containers if Docker is your lab runtime.
**Decode every important part:** `docker ps` lists running containers; `--format` changes columns; `{{...}}` is Docker template syntax.
**Expected observation:** Named containers, or header only when none run.
**If the result differs:** If Docker daemon is down, inspect its service rather than changing Kubernetes.
**Safety / side effect:** Read only; Docker access can confer root-equivalent authority.

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

## Common faults and diagnostic reasoning

- **Confusing a Pod with a VM** — Compare the shared host kernel and Pod network namespace to a guest-kernel VM.

- **OOMKilled** — Check `kubectl describe pod` and memory limits before increasing replicas.
- **Assuming a Pod has one container** — A Pod can have init containers and multiple app containers; check spec and status separately.

## Guided practice and evidence

1. Draw image → runtime → namespaces/cgroups → process.
2. State which namespace makes containers in the same Pod share `localhost`.
3. Observe a disposable container and write down the difference between image and process.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why does changing a Pod memory limit matter even when the node has free RAM?

**Model answer:** A memory limit constrains the container cgroup. The kernel can OOM-kill a process that exceeds that cgroup limit, even when node-wide memory remains available. Requests and limits serve different purposes.

**Official references:** https://kubernetes.io/docs/concepts/workloads/pods/, https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

---

# Deep Lesson 03 — Kubernetes control plane and reconciliation

**Goal:** Trace an API object all the way from the client to a running process, including why etcd, scheduler and kubelet differ.
**Starting point:** You understand programs, containers and Pod vocabulary.
**Hinglish mental model:** `spec` desired state, `status` observed state, controller continuously compares and changes the world.

## Teach the concept from zero

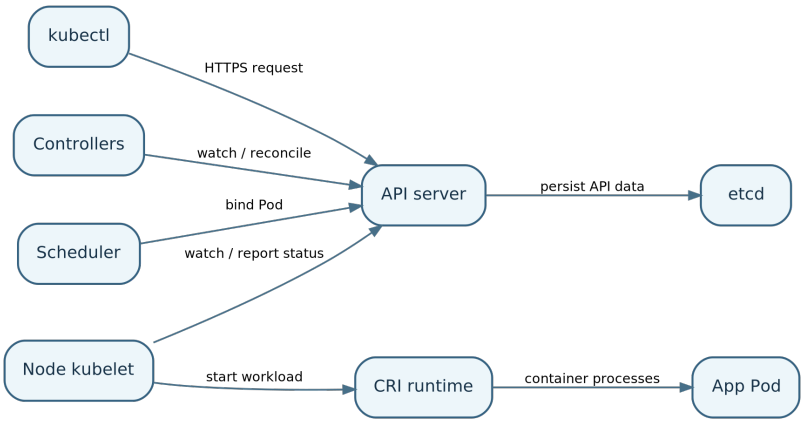
A Kubernetes cluster comprises a **control plane** and one or more **nodes**. The API server is the authenticated, authorized, admission-controlled front door for API object changes. In a typical self-managed cluster, persisted API state is held in **etcd**, a consistent key-value store. The API server persists cluster definitions there; application SQL tables and application logs do not belong in etcd.

When you apply a Deployment, the server stores desired state. A Deployment controller creates/updates a ReplicaSet. A ReplicaSet controller ensures the requested number of matching Pods exists. Pods without an assigned node are eligible for **scheduling**. The scheduler filters nodes against constraints and scores viable candidates, then binds the Pod to a node through the API. The selected node's **kubelet** notices that assignment, instructs the CRI runtime to create the Pod sandbox and containers, performs probes and reports status. The CNI plugin provides Pod networking; CoreDNS answers DNS; Service traffic may be handled by kube-proxy or an alternative implementation.

Think about two different failures. If the API server is temporarily unreachable, existing running containers often keep running, but new desired-state changes and many control loops cannot progress normally. If a kubelet stops, the processes already on that host may continue but their lifecycle/health management is impaired. High availability requires considering each dependency separately.

A reconciliation loop is **level-based**, not a one-time script. Controllers observe the latest current state and keep acting toward desired state. Deleting one Pod from a Deployment does not permanently reduce replicas; the controller creates a replacement. Deleting a standalone Pod does not automatically create a new one because there is no higher-level controller. This single distinction explains a huge amount of Kubernetes behavior.

**Figure 3: Kubernetes control plane and reconciliation**




**How to read this diagram, arrow by arrow:**

- **kubectl → API server** (HTTPS request): kubectl performs client-side work then communicates with the API server; it does not invoke the kubelet directly.

- **API server → etcd** (persist API data): etcd records API state, not arbitrary app files or logs.
- **Controllers → API server** (watch / reconcile): Controllers use watches/lists and submit changes such as ReplicaSet or Pod creation.
- **Scheduler → API server** (bind Pod): Scheduler assigns a suitable node by recording binding via the API.
- **Node kubelet → API server** (watch / report status): Kubelet sees Pod assignments and posts Pod/node status; it is not the scheduler.
- **Node kubelet → CRI runtime** (start workload): Kubelet uses CRI APIs to manage containers and sandboxes.
- **CRI runtime → App Pod** (container processes): The runtime starts Linux processes; that is the workload, not an API server execution.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 11: `kubectl config current-context`

**Where:** HOST
**Why run it:** Prevent accidental changes to an unrelated cluster.
**Decode every important part:** `kubectl config` handles kubeconfig; `current-context` prints active context.
**Expected observation:** For our kind cluster it is usually `kind-artha-lab`.
**If the result differs:** Stop if a production or unknown context appears.
**Safety / side effect:** Read only; make checking context a habit.

```
kubectl config current-context
```

### Command 12: `kubectl cluster-info`

**Where:** HOST
**Why run it:** Discover API endpoint and system-service information.
**Decode every important part:** `cluster-info` is a kubectl subcommand; no namespace or resource needed.
**Expected observation:** Control-plane endpoint and possible CoreDNS URL.
**If the result differs:** If unavailable, inspect VPN, credentials, server endpoint and kubeconfig.
**Safety / side effect:** Read only.

```
kubectl cluster-info
```

### Command 13: `kubectl get nodes -o wide`

**Where:** HOST
**Why run it:** List schedulable node identities and versions.
**Decode every important part:** `get` reads API objects; `nodes` is a cluster-scoped kind; `-o wide` adds fields.
**Expected observation:** NAME, STATUS, ROLES, AGE, VERSION, INTERNAL-IP.
**If the result differs:** Node NotReady is not equal to an API-server failure; inspect that node.
**Safety / side effect:** Read only.

```
kubectl get nodes -o wide
```

### Command 14: `kubectl get pods -n kube-system`

**Where:** HOST
**Why run it:** See system workloads such as DNS and networking.
**Decode every important part:** `-n` selects the namespace; `kube-system` contains system Pods.
**Expected observation:** A list of system Pods; components vary with distribution.
**If the result differs:** In managed clusters control-plane Pods are often not visible to customers.
**Safety / side effect:** Read only.

```
kubectl get pods -n kube-system
```

### Command 15: `kubectl get --raw /readyz`

**Where:** HOST
**Why run it:** Check API server readiness endpoint.
**Decode every important part:** `get --raw` makes a raw request to the API path; `/readyz` is an API health endpoint.
**Expected observation:** Usually `ok` when ready and authorized.
**If the result differs:** Failures can indicate transport, TLS, authentication, authorization or server health.
**Safety / side effect:** Read only; do not assume endpoint exposure policy.

```
kubectl get --raw /readyz
```

## Common faults and diagnostic reasoning

- **Deployment has 2 desired but 1 ready** — Check Deployment conditions, ReplicaSet, Pods, Events and readiness before touching components.
- **Scheduler not running** — Previously bound Pods may continue; new Pending Pods do not get placed.
- `kubectl` **works but Services fail** — API control-path health does not prove dataplane CNI/DNS health.

## Guided practice and evidence

1. Sketch all eight boxes from memory and explain arrows without looking.
2. Inspect namespace, deployments and system Pods; identify which commands read state.
3. Delete a disposable Deployment-managed Pod, watch replacement, and compare with standalone Pod deletion.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Walk me through `kubectl apply -f deployment.yaml` from file to running process.

**Model answer:** The client sends a request to the API server. After admission and persistence, Deployment and ReplicaSet controllers reconcile desired Pods. The scheduler binds new Pods to nodes. Node kubelets ask the runtime to start containers, then report status. Network and Service implementations enable communication. None of those components is equivalent to the API server.

**Official references:** https://kubernetes.io/docs/concepts/architecture/, https://kubernetes.io/docs/concepts/architecture/controller/

# Deep Lesson 04 — Pods, metadata, labels, selectors and object lifecycle

**Goal:** Read Pod YAML, predict Pod phases, and know exactly why selectors matter.
**Starting point:** A local kind cluster and namespace `artha-lab` are available.
**Hinglish mental model:** Labels tags hain; selectors un tags ke basis par matching Pods dhundhte hain.

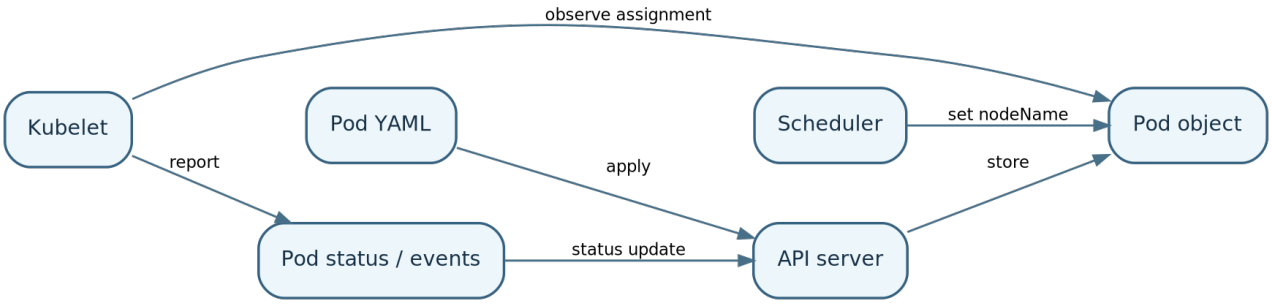
## Teach the concept from zero

Every Kubernetes API object has a kind, an API version, and metadata. Names are unique within their API scope; most workload kinds are namespaced, while Nodes and Namespaces are cluster-scoped. Labels are indexed identifying key/value pairs used by selectors, while annotations carry non-identifying metadata. A Pod manifest says which containers belong together and how to run them. `kubectl get` displays a summary; `kubectl describe` shows detailed conditions and Events; `kubectl get -o yaml` shows an API representation including status and server-created metadata.

Pod **phase** is high-level (Pending, Running, Succeeded, Failed, Unknown). It is not the same as a container's state (Waiting/Running/Terminated) or a reason such as `CrashLoopBackOff` or `ImagePullBackOff`. `Pending` can mean unscheduled or waiting on image/volume/sandbox setup; inspect the actual Conditions and Events. `Ready` is a Pod condition that captures whether appropriate containers are ready to serve, not whether the process exists. A Pod IP is normally ephemeral and should not be hardcoded as a durable service endpoint.

A standalone Pod is fine for education and certain special workflows but not usually for long-lived application replicas. Deployments and StatefulSets provide higher-level ownership. `ownerReferences` is metadata that lets you trace which controller created an object. `kubectl delete pod` does not delete its controller; it often causes replacement. Be comfortable reading resource relationships before editing live state.

**Figure 4: Pods, metadata, labels, selectors and object lifecycle**




**How to read this diagram, arrow by arrow:**

- **Pod YAML → API server** (apply): Client submits an API description after selecting correct context.
- **API server → Pod object** (store): The Pod object stores desired container information.
- **Scheduler → Pod object** (set nodeName): Scheduler binds to a node; it does not choose individual containers.
- **Kubelet → Pod object** (observe assignment): Kubelet watches Pods bound to its node.
- **Kubelet → Pod status / events** (report): Kubelet reports phases, conditions and per-container states.
- **Pod status / events → API server** (status update): Status is updated in the API and can be inspected independently of spec.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 16: `kubectl apply -f manifests/01-pod.yaml`

**Where:** HOST in unzipped original lab pack
**Why run it:** Create/update the demonstration Pod declaratively.
**Decode every important part:** `apply` reconciles file declarative config; `-f` reads a path.
**Expected observation:** A creation/configuration message; Pod may take time to become Running.
**If the result differs:** Use `kubectl get events` if validation, image pull or scheduling fails.
**Safety / side effect:** Mutates disposable namespace; check context first.

```
kubectl apply -f manifests/01-pod.yaml
```

### Command 17: `kubectl get pods -n artha-lab -o wide`

**Where:** HOST
**Why run it:** See phase, readiness, restarts and node.
**Decode every important part:** `get pods` lists Pods; `-o wide` adds Pod IP and node.
**Expected observation:** Standalone Pod with 1/1 Ready once image is available.
**If the result differs:** READY may remain 0/1 during pull or probes; inspect details.
**Safety / side effect:** Read only.

```
kubectl get pods -n artha-lab -o wide
```

### Command 18: `kubectl describe pod standalone-nginx -n artha-lab`

**Where:** HOST
**Why run it:** Understand observed lifecycle and Events.
**Decode every important part:** `describe` prints human-friendly object detail; name selects one Pod; `-n` selects scope.
**Expected observation:** Containers, scheduling decisions, conditions, Events at bottom.
**If the result differs:** No Events is normal for a healthy Pod; do not infer error from absence.
**Safety / side effect:** Read only.

```
kubectl describe pod standalone-nginx -n artha-lab
```

### Command 19: `kubectl get pod standalone-nginx -n artha-lab -o jsonpath="{.spec.nodeName}"`

**Where:** HOST
**Why run it:** Extract the chosen node from a JSONPath field.
**Decode every important part:** `-o jsonpath` selects structured output; `{.spec.nodeName}` navigates desired/bound node.
**Expected observation:** Node name with no trailing newline in this variant.
**If the result differs:** Blank means unscheduled; inspect Pod conditions and events.
**Safety / side effect:** Read only.

```
kubectl get pod standalone-nginx -n artha-lab -o jsonpath="{.spec.nodeName}"
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  namespace: artha-lab
  labels:
    app: hello
spec:
  containers:
    - name: web
      image: nginx:1.27
      ports:
        - containerPort: 80
```

**Field-by-field explanation:**

- `apiVersion: v1`: use the core API group for Pods.
- `kind: Pod`: ask the API to create a Pod object, not a Deployment.
- `metadata`: identity information, labels and annotations live here.
- `name`: unique Pod name in the namespace.
- `namespace`: tells the API which namespace owns this object.
- `labels.app`: selector-friendly key/value marker; it has no magic routing behavior by itself.
- `spec`: desired runtime configuration; Kubernetes updates status separately.
- `containers`: an array; each `- name` begins one container.
- `image`: repository and image tag. Prefer reviewed immutable digests in production.
- `ports.containerPort`: descriptive container port; it does not expose traffic outside the Pod by itself.

**Change exercise:** Change `app: hello` to `app: demo` and observe which selector queries find the Pod.

## Common faults and diagnostic reasoning

- **ImagePullBackOff** — First check image and tag, registry DNS, credentials and network; backoff is a symptom.
- **Pod remains Pending** — Inspect insufficient resources, taints/affinity, unbound PVC and node readiness.
- **Confusing Ready with Running** — A container can be running while readiness fails, thus receiving no Service traffic.

## Guided practice and evidence

1. Apply a standalone Pod in the training namespace and capture `get`, `describe` and JSONPath outputs.
2. Change a harmless Pod label and find it with `-l exercise=pod`.
3. Delete the Pod; explain why it stays deleted without a controller.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** When should I use `describe` instead of only `get`?

**Model answer:** `get` quickly summarizes status; `describe` adds detailed spec, status conditions and Events that explain scheduling, image, volume and probe failures.

**Official references:** https://kubernetes.io/docs/concepts/workloads/pods/, https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/

---

# Deep Lesson 05 — Deployments, ReplicaSets, rolling updates and safe rollback

**Goal:** Model reconciliation, availability constraints and rollout strategies rather than memorizing commands.
**Starting point:** Know Pods, labels and readiness.
**Hinglish mental model:** Deployment = target recipe, ReplicaSet = number maintainer, Pod = running instance.

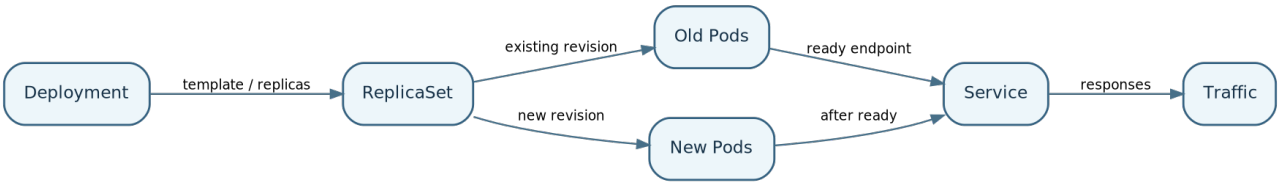
## Teach the concept from zero

A **Deployment** manages ReplicaSets, which manage matching Pods. The desired count is specified by `.spec.replicas`. During image or template changes, a new ReplicaSet may be created, and Pods are gradually replaced according to the deployment strategy. `maxSurge` is how many Pods may exist above the desired replica count during a rolling update; `maxUnavailable` is how many may be unavailable. With desired replicas of 2, `maxSurge: 1`, `maxUnavailable: 0`, Kubernetes may temporarily run 3 Pods while preserving 2 available, provided capacity and readiness permit.

An update is not automatically successful just because the new Pod reaches Running. It must satisfy readiness and whatever application/business SLO tests your release gate requires. If a new image tag is nonexistent, a new Pod may remain in ImagePullBackOff and rollout can stall. A rollback returns the Deployment Pod template to a previous revision; it may still fail if the previous image vanished or config has changed. `kubectl rollout undo` is a recovery tool, not a version-control replacement. Changes to a Deployment's `.spec.template` drive revisions; a pure replica change is not normally a new rollout revision.

Race conditions matter. A Deployment may be progressing while Services are serving mixed versions. Database migrations that are not backward compatible can break an otherwise healthy rolling update. Always plan compatibility, health checks, Pod termination, graceful shutdown and minimum spare capacity. In large systems, use canaries with traffic shaping and measured error rates in addition to the basic Deployment rolling update.

**Figure 5: Deployments, ReplicaSets, rolling updates and safe rollback**




**How to read this diagram, arrow by arrow:**

- **Deployment → ReplicaSet** (template / replicas): Deployment controller owns RS versions and rollout strategy.
- **ReplicaSet → Old Pods** (existing revision): Old RS owns its old-revision Pods until rollout progresses.
- **ReplicaSet → New Pods** (new revision): New RS creates replacement Pods; true rollout has distinct RS objects.
- **Old Pods → Service** (ready endpoint): Only Ready matching Pods normally become usable Service backends.
- **New Pods → Service** (after ready): New Pods join Service traffic when ready; actual dataplane depends on provider.

- **Service → Traffic** (responses): Service gives clients stable discovery, not a guarantee that app responses are correct.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 20: `kubectl apply -f manifests/02-deployment.yaml`

**Where:** HOST / unzipped lab
**Why run it:** Create a two-replica Deployment.
**Decode every important part:** `apply -f` submits a declarative file containing `apps/v1` Deployment.
**Expected observation:** Deployment, ReplicaSet and two Pods appear.
**If the result differs:** If replicas stuck, describe Deployment and list Pods/events.
**Safety / side effect:** Mutates the lab namespace.

```
kubectl apply -f manifests/02-deployment.yaml
```

### Command 21: `kubectl rollout status deployment/web -n artha-lab --timeout=180s`

**Where:** HOST
**Why run it:** Wait for rollout readiness with a bounded timeout.
**Decode every important part:** `rollout status` monitors Deployment progression; `--timeout=180s` stops waiting after 180 seconds.
**Expected observation:** `successfully rolled out` when ready.
**If the result differs:** Timeout means inspect diagnostics; do not assume no Pods are running.
**Safety / side effect:** Read only/wait.

```
kubectl rollout status deployment/web -n artha-lab --timeout=180s
```

### Command 22: `kubectl get deployment,replicaset,pod -n artha-lab`

**Where:** HOST
**Why run it:** Trace ownership chain.
**Decode every important part:** Comma-separated types ask for multiple resource kinds in one list.
**Expected observation:** A deployment, one or more ReplicaSets and Pods.
**If the result differs:** An old RS with 0 replicas may remain for history.
**Safety / side effect:** Read only.

```
kubectl get deployment,replicaset,pod -n artha-lab
```

### Command 23: `kubectl scale deployment/web --replicas=3 -n artha-lab`

**Where:** HOST
**Why run it:** Change desired replica count deliberately.
**Decode every important part:** `scale` modifies replicas; `--replicas=3` sets count; `-n` namespace.
**Expected observation:** Eventually three running/ready Pods if capacity exists.
**If the result differs:** If Pending inspect node allocatable vs requests.
**Safety / side effect:** Changes workload capacity, even in lab; use only disposable namespace.

```
kubectl scale deployment/web --replicas=3 -n artha-lab
```

### Command 24: `kubectl set image deployment/web nginx=nginx:invalid-artha-lab-tag -n artha-lab`

**Where:** HOST
**Why run it:** Cause a reversible test image-pull failure.
**Decode every important part:** `set image` updates container named nginx to a new image reference.
**Expected observation:** New Pod(s) cannot pull; rollout should not become fully healthy.
**If the result differs:** Use Events; confirm the container name is exactly `nginx`.
**Safety / side effect:** Intentional failure; ONLY in disposable namespace.

```
kubectl set image deployment/web nginx=nginx:invalid-artha-lab-tag -n artha-lab
```

### Command 25: `kubectl rollout undo deployment/web -n artha-lab`

**Where:** HOST
**Why run it:** Revert the bad Pod-template revision.
**Decode every important part:** `undo` reverts to previous Deployment revision if retained.
**Expected observation:** Working Pod template restored and rollout eventually healthy.
**If the result differs:** Check `rollout history` and image references if undo fails.
**Safety / side effect:** Mutates lab Deployment; never use unreviewed in production.

```
kubectl rollout undo deployment/web -n artha-lab
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: artha-lab
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
```

**Field-by-field explanation:**

- `apps/v1`: the stable Deployment API group/version.
- `replicas: 2`: desired Pod count.
- `strategy.type`: RollingUpdate rather than Recreate.
- `maxUnavailable: 0`: do not intentionally reduce available replicas during update.
- `maxSurge: 1`: allow one extra Pod above desired count temporarily.

- `selector.matchLabels.app`: immutable selector that identifies owned Pods; it must match the template labels.
- `template`: Pod blueprint; changing its specification can trigger a new ReplicaSet.
- `template.metadata.labels`: labels on created Pods, used for selectors.
- `containers` and `image`: runtime instruction for each Pod.
- `containerPort: 80`: metadata about listened container port; no network exposure by itself.

**Change exercise:** Change one image to a known available tag and predict which ReplicaSets should exist during rollout.

## Common faults and diagnostic reasoning

- **Rollout complete but errors rising** — Kubernetes readiness is not enough; use smoke tests and error budget signals.
- **MaxSurge saturates capacity** — Plan node spare room or adjust strategies; overly aggressive limits may stall rollouts.
- **Rollback does not restore database** — Data migrations need separately tested backup/compatibility strategies.

## Guided practice and evidence

1. Deploy 2 replicas; show the selected Pods and matching labels.
2. Scale to 3 then 2 and write how RS reconciliation reacts.
3. Introduce an invalid image tag, capture Events, roll back and show restored Ready count.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** What is the difference between a Deployment, ReplicaSet and Pod?

**Model answer:** Deployment owns rollout strategy and revision history. ReplicaSet maintains a count of matching Pods for one Pod template. Pod is the assigned runtime environment containing application containers.

**Official references:** https://kubernetes.io/docs/concepts/workloads/controllers/deployment/, https://kubernetes.io/docs/tasks/run-application/update-deployment-rolling/

---

# Deep Lesson 06 — Services, DNS, EndpointSlices and HTTP request flow

**Goal:** Trace each hop from DNS lookup to healthy Pod without assuming a single dataplane implementation.
**Starting point:** A Deployment with `app: web` Pods exists.
**Hinglish mental model:** Pod IP badalta hai; Service ka name stable discovery point hai.

## Teach the concept from zero

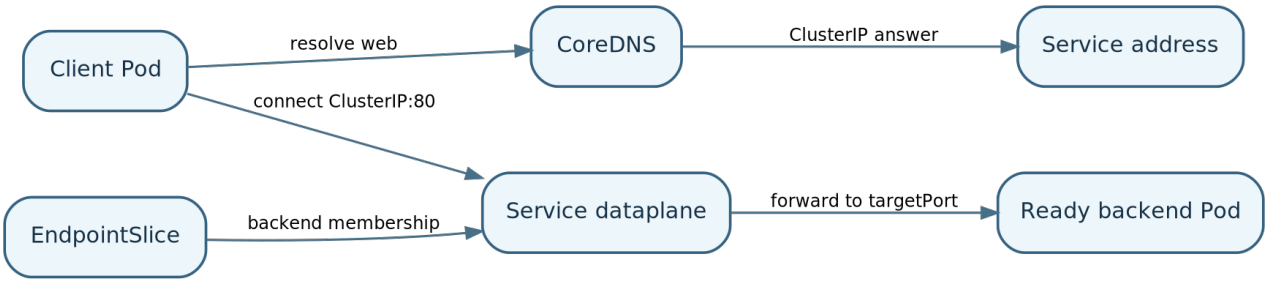
Pods have IPs that can change as they are recreated. A **Service** defines a stable virtual endpoint and an intent to route to a set of Pods selected by labels. A ClusterIP Service is typically only routable within the cluster network. A Service's `selector` matches labels; it is not a Pod name. **EndpointSlice** objects list eligible backend addresses and useful readiness/port conditions. If selectors are wrong, the Service may exist but have zero usable backends.

A Pod asking for `web.artha-lab.svc.cluster.local` normally queries cluster DNS (commonly CoreDNS). It gets the Service address, not a permanent list of Pod IPs for an ordinary ClusterIP Service. The request then traverses the actual Service dataplane implementation—often kernel rules managed by kube-proxy, but some CNIs use eBPF or other alternatives—to a selected backend Pod. Port layers differ: `Service.port` is the port clients target; `Service.targetPort` resolves to a container port number or named port. `containerPort` in Pod spec is not an automatic firewall rule.

**Ingress** is an HTTP routing API backed by an installed controller. **Gateway API** is a more expressive, role-aware approach for traffic policies, with implementations supplied by controllers. Neither API creates a working external route until controllers and infrastructure exist. **NodePort** exposes a port range on nodes and is not equivalent to production-grade external TLS and security. For debugging, split DNS, Service selection, endpoints, and direct Pod reachability into independent steps.

`kubectl port-forward service/web 8080:80` gives your local machine a path to one selected Pod by port-forwarding; it is a useful teaching shortcut but is not evidence that the complete external ingress dataplane works.

**Figure 6: Services, DNS, EndpointSlices and HTTP request flow**




**How to read this diagram, arrow by arrow:**

- **Client Pod → CoreDNS** (resolve web): Resolver asks for the Service name, often expanded with search domains.
- **CoreDNS → Service address** (ClusterIP answer): CoreDNS replies with the Service address when the name is valid.
- **Client Pod → Service dataplane** (connect ClusterIP:80): The client sends packets to the virtual IP and port; DNS resolution alone does not make them flow.

- **EndpointSlice → Service dataplane** (backend membership): EndpointSlices inform which Pod IPs/ports are eligible for forwarding.
- **Service dataplane → Ready backend Pod** (forward to targetPort): Dataplane chooses an eligible backend and sends traffic to that Pod address/port.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 26: `kubectl apply -f manifests/03-service.yaml`

**Where:** HOST / unzipped lab
**Why run it:** Create a ClusterIP Service selecting `app: web`.
**Decode every important part:** Service spec defines `selector`, `port`, and `targetPort`; `apply -f` submits it.
**Expected observation:** Service gets a ClusterIP and port.
**If the result differs:** Check selector/namespace if endpoints are empty.
**Safety / side effect:** Mutates lab namespace.

```
kubectl apply -f manifests/03-service.yaml
```

### Command 27: `kubectl get service web -n artha-lab -o wide`

**Where:** HOST
**Why run it:** Inspect address and exported ports.
**Decode every important part:** `service` can abbreviate to `svc`; `-o wide` may show additional fields.
**Expected observation:** ClusterIP, `80/TCP`, selector summary.
**If the result differs:** An address here does not prove working endpoints.
**Safety / side effect:** Read only.

```
kubectl get service web -n artha-lab -o wide
```

### Command 28: `kubectl get endpointslices -n artha-lab -l kubernetes.io/service-name=web`

**Where:** HOST
**Why run it:** Find the Service backend population.
**Decode every important part:** `-l key=value` is a label-selector filter; the label connects slices to a Service.
**Expected observation:** One or more slices with addresses when Pods match and are ready.
**If the result differs:** If none, compare Service selectors and Pod labels/readiness.
**Safety / side effect:** Read only.

```
kubectl get endpointslices -n artha-lab -l kubernetes.io/service-name=web
```

### Command 29: `kubectl get pods -n artha-lab --show-labels`

**Where:** HOST
**Why run it:** Check matching labels on real Pods.
**Decode every important part:** `--show-labels` adds label sets to each row.
**Expected observation:** `app=web` on the intended backend Pods.
**If the result differs:** If labels disagree, fix declarative specs; avoid ad-hoc production patches.
**Safety / side effect:** Read only.

```
kubectl get pods -n artha-lab --show-labels
```

### Command 30: `kubectl port-forward -n artha-lab service/web 8080:80`

**Where:** HOST terminal A
**Why run it:** Temporarily expose a Service-selected Pod to localhost.
**Decode every important part:** `port-forward` tunnels through Kubernetes API; `8080:80` maps host TCP 8080 to Service port 80.
**Expected observation:** Terminal waits and prints Forwarding messages.
**If the result differs:** Port 8080 conflict? choose another local port.
**Safety / side effect:** Runs a temporary local listener until Ctrl+C; not a production route.

```
kubectl port-forward -n artha-lab service/web 8080:80
```

### Command 31: `curl -i http://127.0.0.1:8080`

**Where:** HOST terminal B while forward runs
**Why run it:** Make a simple HTTP request through the tunnel.
**Decode every important part:** `curl` is HTTP client; `-i` displays response headers; loopback IP refers to this host.
**Expected observation:** HTTP status and nginx response body (image dependent).
**If the result differs:** Connection refused usually means port-forward exited; 503-like problems suggest service/backend.
**Safety / side effect:** Read only against the local tunnel.

```
curl -i http://127.0.0.1:8080
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: artha-lab
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
  - name: http
    protocol: TCP
    port: 80
    targetPort: 80
```

**Field-by-field explanation:**

- `kind: Service`: creates stable discovery and traffic abstraction.
- `type: ClusterIP`: internal Service address (not internet exposed by default).
- `selector.app: web`: includes Pods labeled `app=web`; not Pod name matching.
- `ports`: list supports multiple ports.
- `name: http`: meaningful identifier for this Service port.
- `protocol: TCP`: layer-4 transport.
- `port: 80`: clients connect to Service port 80.
- `targetPort: 80`: dataplane routes to port 80 on eligible Pods.

**Change exercise:** Set Service selector to `app: does-not-exist`; explain why DNS may still resolve but traffic has no endpoints.

## Common faults and diagnostic reasoning

- **DNS returns IP but requests fail** — Check Service endpoints, policy, routing and server listening port.
- **No EndpointSlice backends** — Selector mismatch or Pod readiness may cause this; inspect both.
- **NetworkPolicy appears ignored** — Enforcement requires a suitable CNI; a Policy API object alone does not block traffic.

## Guided practice and evidence

1. Write client→DNS→ClusterIP→dataplane→Pod request path without skipping EndpointSlice.
2. Intentionally change a lab Service selector to a nonexistent label; capture no-endpoint evidence, then restore.
3. Trace `port`, `targetPort` and Pod `containerPort` values for the training web Service.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** How would you debug `curl: connection timed out` to a ClusterIP Service?

**Model answer:** Check DNS resolution, Service IP/ports, selectors and EndpointSlices, Pod readiness/listening port, NetworkPolicy and CNI routing/dataplane. Separate timeouts from HTTP errors and compare direct Pod access as an authorized diagnostic.

**Official references:** https://kubernetes.io/docs/concepts/services-networking/service/, https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/, https://gateway-api.sigs.k8s.io/

---

# Deep Lesson 07 — ConfigMaps, Secrets and configuration lifecycle

**Goal:** Separate image code from environment configuration, secret delivery, and safe rotation.
**Starting point:** You understand basic Pod specs and namespaces.
**Hinglish mental model:** Image same reh sakti hai, environment config alag hota hai; secret base64 hone se encrypted nahi ho jaata.

## Teach the concept from zero

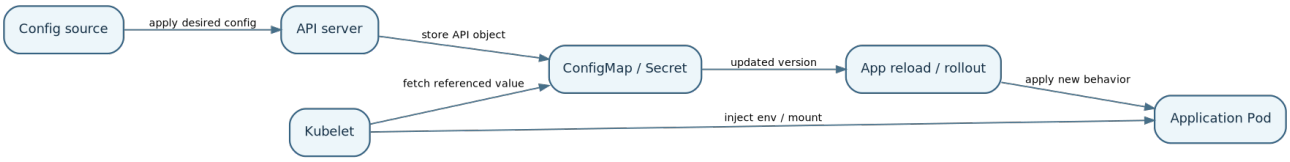
A **ConfigMap** stores non-confidential key/value configuration for workloads. A **Secret** stores sensitive-ish API object data but is **not automatically encrypted end-to-end merely because values are base64-encoded**. Whether Secrets are encrypted at rest in etcd depends on cluster configuration; access to Secret objects and volume projections must be restricted. In production, consider external secret managers, short-lived workload identity, audit trails and rotation.

There are common injection patterns. `env` sets an individual environment variable from a ConfigMap or Secret key. `envFrom` imports many keys as environment variables. `volumes` can project keys into files. **Environment variables are fixed when the container starts**; updating a ConfigMap does not magically update running environment variables. Mounted ConfigMap/Secret volumes can update eventually when projected, subject to kubelet refresh timing and application reload behavior; a `subPath` mount has special update limitations. Your application may need a SIGHUP/reload or safe rollout even when the file contents change.

Security trade-off: avoid writing secrets into Git, container images, CI logs, shell history, stdout, or debug screenshots. A workload that can read a secret can often leak it through application code; RBAC, Pod hardening and least-privilege workload identity are all complementary. In labs use obviously fake values and never demonstrate by printing real token contents. Changes to secret material should have rollback and expiration stories, not just a successful `kubectl apply`.

Namespacing matters: a Pod's ConfigMap/Secret references resolve in its own namespace. `metadata.name` and `namespace` must match. A missing key can prevent startup depending on `optional` flags. For larger teams, GitOps-managed config and policy validation help keep changes controlled.

**Figure 7: ConfigMaps, Secrets and configuration lifecycle**




**How to read this diagram, arrow by arrow:**

- **Config source → API server** (apply desired config): A user or GitOps reconciler submits a configuration object.
- **API server → ConfigMap / Secret** (store API object): Kubernetes stores key/value API objects; Secret protection depends on configured encryption and access controls.
- **Kubelet → ConfigMap / Secret** (fetch referenced value): The node agent resolves references for Pods placed on its node.
- **Kubelet → Application Pod** (inject env / mount): The runtime starts the container with env values or mounted files.

- **ConfigMap / Secret → App reload / rollout** (updated version): Config change alone does not guarantee that a running app has consumed new values.
- **App reload / rollout → Application Pod** (apply new behavior): The application re-reads a file, restarts, or is rolled out deliberately.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 32: `kubectl apply -f manifests/04-configmap.yaml`

**Where:** HOST / unzipped lab
**Why run it:** Create a non-sensitive ConfigMap and fake Secret.
**Decode every important part:** `apply -f` accepts a multi-document YAML separated by `---`.
**Expected observation:** Objects `demo-config` and `demo-secret` appear in artha-lab.
**If the result differs:** Use `get configmap` and `get secret` names; avoid printing values unnecessarily.
**Safety / side effect:** Mutates only training objects; dummy data only.

```
kubectl apply -f manifests/04-configmap.yaml
```

### Command 33: `kubectl get configmap demo-config -n artha-lab`

**Where:** HOST
**Why run it:** Confirm object existence and key count.
**Decode every important part:** `get configmap NAME` reads one namespaced config object.
**Expected observation:** DATA count; not necessarily full key values.
**If the result differs:** NotFound means wrong name or namespace.
**Safety / side effect:** Read only.

```
kubectl get configmap demo-config -n artha-lab
```

### Command 34: `kubectl apply -f manifests/05-configured-pod.yaml`

**Where:** HOST / unzipped lab
**Why run it:** Inject config into a disposable Pod.
**Decode every important part:** Manifest has an `env.valueFrom.configMapKeyRef` and `envFrom.secretRef` reference.
**Expected observation:** Pod eventually Running if image available.
**If the result differs:** Bad key/name may show `CreateContainerConfigError`.
**Safety / side effect:** Mutates lab Pod; do not print secrets.

```
kubectl apply -f manifests/05-configured-pod.yaml
```

### Command 35: `kubectl describe pod configured -n artha-lab`

**Where:** HOST
**Why run it:** Inspect references and lifecycle without dumping Secret bytes.
**Decode every important part:** `describe` includes Env references and Events; it may show metadata about env.
**Expected observation:** ConfigMap/Secret references; normal Events on success.
**If the result differs:** If missing, confirm namespace/name/key.
**Safety / side effect:** Read only; output can still contain some metadata, review sharing.

```
kubectl describe pod configured -n artha-lab
```

### Command 36: `kubectl logs configured -n artha-lab`

**Where:** HOST
**Why run it:** Inspect non-sensitive debug log from lab Pod.
**Decode every important part:** `logs` reads container stdout/stderr captured by runtime and kubelet.
**Expected observation:** Example shows `APP_MODE=training`, not Secret token.
**If the result differs:** If no logs, inspect Pod/container name and status.
**Safety / side effect:** Read only, but application logs may contain sensitive content in real systems.

```
kubectl logs configured -n artha-lab
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-settings
  namespace: artha-lab
data:
  APP_MODE: "training"
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Pod
metadata:
  name: env-demo
  namespace: artha-lab
spec:
  containers:
  - name: app
    image: busybox:1.36
    command: ["sh", "-c", "echo $APP_MODE; sleep 3600"]
    env:
    - name: APP_MODE
      valueFrom:
        configMapKeyRef:
          name: app-settings
          key: APP_MODE
```

**Field-by-field explanation:**

- `ConfigMap.data`: values are strings; use quotes for values that resemble booleans.
- `---`: document separator; a single file can define two API objects.
- `Pod.metadata.namespace`: reference lookup is within this namespace.
- `command`: container entry command array; shell expands `$APP_MODE` because `sh -c` is used.
- `env[].name`: environment variable name inside container.
- `valueFrom.configMapKeyRef`: look up one ConfigMap value instead of hardcoding it.
- `name: app-settings`: must match ConfigMap name.
- `key: APP_MODE`: must exist in ConfigMap data unless optional behavior is declared.

**Change exercise:** Change the ConfigMap while leaving the Pod running; predict what its process environment contains.

## Common faults and diagnostic reasoning

- **Missing ConfigMap key** — Check `configMapKeyRef.key`, ConfigMap `data` and Pod namespace.

- **Changed config but app unchanged** — Environment variable values need restart; mounted files may update, but app reload is separate.
- **Secrets visible to cluster administrators** — Base64 is encoding, not encryption; configure etcd encryption, audit and least-privilege RBAC.

## Guided practice and evidence

1. Inspect the fake ConfigMap and explain which keys will enter the container.
2. Change APP_MODE in a ConfigMap, explain why an existing environment variable did not change.
3. Document a safe config rollout and a secret-rotation checklist without exposing a real token.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why is a Kubernetes Secret not a complete secret-management system?

**Model answer:** Secret content uses an API object whose confidentiality depends on RBAC, at-rest encryption, workload permissions, backups and logging. Base64 is not encryption. A full solution adds short-lived access, delivery controls, rotation and auditing.

**Official references:** https://kubernetes.io/docs/concepts/configuration/configmap/, https://kubernetes.io/docs/concepts/configuration/secret/

---

# Deep Lesson 08 — Readiness, liveness, startup probes and graceful shutdown

**Goal:** Design health signals that protect users rather than accidentally restarting healthy processes.
**Starting point:** You can deploy a web app and read `describe` Events.
**Hinglish mental model:** Readiness = traffic allowed? Liveness = restart needed? Startup = shuru hone ke liye time do.

## Teach the concept from zero

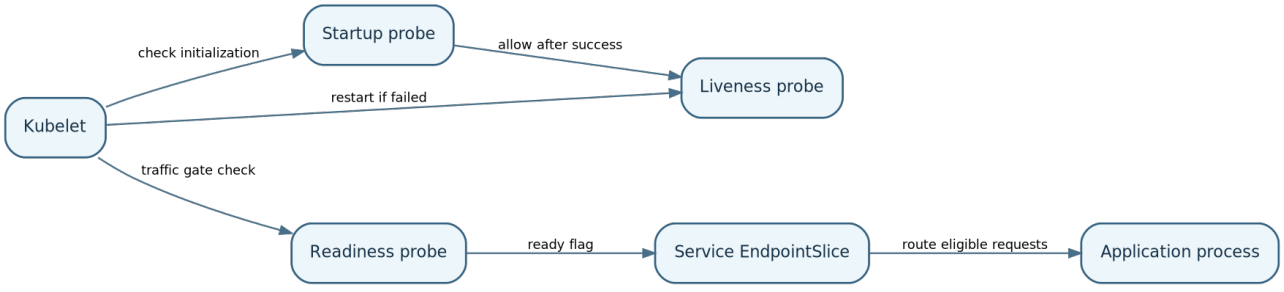
Three health concepts solve different problems. A **startup probe** gates startup checks for applications that initialize slowly. A **liveness probe** asks whether a running container is broken enough to need restarting; excessive liveness sensitivity can induce restart loops. A **readiness probe** asks whether the process can accept intended traffic now; a failing readiness check removes an endpoint from normal Service routing without itself restarting a container. An ordinary Pod condition being Running does not mean its app is ready to serve traffic.

Use a cheap health endpoint that accurately reflects the relevant failure domain. `GET /health/live` might indicate internal event-loop progress. `GET /health/ready` can include dependencies if a dependency outage should stop this instance taking requests; however, making every replica unready because of a shared, optional downstream can worsen the outage. Set `timeoutSeconds`, `periodSeconds`, `failureThreshold`, and initial delays based on startup and latency characteristics, not by copying random defaults. Probe checking is performed by kubelet; HTTP response codes from 200 through 399 count as successful for HTTP probes.

Shutting down gracefully matters as much as starting. A Pod deletion sets a deletion timestamp, endpoints are updated, the runtime normally receives SIGTERM, and after a grace period SIGKILL may force termination. Readiness, EndpointSlice propagation, load-balancer delay and in-flight request handling interact; account for terminationGracePeriodSeconds and app signal handlers. **PodDisruptionBudget (PDB)** restricts certain voluntary evictions via the eviction API, not unexpected node deaths. Do not claim PDB protects against all outages.

Debug method: identify the actual condition, read Events, inspect container previous logs, compare health endpoint from inside the Pod, and change only one variable in the disposable lab. An invalid readiness path is a powerful safe demonstration because it creates a Running-but-NotReady process.

**Figure 8: Readiness, liveness, startup probes and graceful shutdown**




**How to read this diagram, arrow by arrow:**

- **Kubelet → Startup probe** (check initialization): Startup probe is used during initialization; it defers other probes until successful.
- **Startup probe → Liveness probe** (allow after success): After startup passes, liveness monitoring becomes active.

- **Kubelet → Liveness probe** (restart if failed): Repeated liveness failures cause container restart according to restart policy.
- **Kubelet → Readiness probe** (traffic gate check): Readiness is evaluated repeatedly while running.
- **Readiness probe → Service EndpointSlice** (ready flag): Readiness influences which endpoint entries are eligible for normal Service traffic.
- **Service EndpointSlice → Application process** (route eligible requests): Traffic is directed to a ready app backend; endpoint updates are not instantaneous everywhere.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 37: `kubectl apply -f manifests/06-readiness-failure.yaml`

**Where:** HOST / lab pack
**Why run it:** Create a Pod with a deliberately missing readiness endpoint.
**Decode every important part:** One file contains Deployment and Service separated by `---`.
**Expected observation:** Pod often shows Running but READY 0/1.
**If the result differs:** If Pending inspect image pull or scheduling before probe behavior.
**Safety / side effect:** Intentional failure only in practice cluster.

```
kubectl apply -f manifests/06-readiness-failure.yaml
```

### Command 38: `kubectl get pods -n artha-lab -l app=readiness-demo -w`

**Where:** HOST
**Why run it:** Watch readiness transition as kubelet checks the path.
**Decode every important part:** `-l` filters by label; `-w` streams watch updates until Ctrl+C.
**Expected observation:** Pod remains 0/1 Ready while container can remain Running.
**If the result differs:** If no Pods, inspect Deployment or selector.
**Safety / side effect:** Read only; long-running until interrupted.

```
kubectl get pods -n artha-lab -l app=readiness-demo -w
```

### Command 39: `kubectl describe deployment readiness-demo -n artha-lab`

**Where:** HOST
**Why run it:** See Deployment availability/progress.
**Decode every important part:** `describe deployment NAME` summarizes replicas, conditions and events.
**Expected observation:** Available replica count remains below desired.
**If the result differs:** Check Pod `describe` for precise probe failure messages.
**Safety / side effect:** Read only.

```
kubectl describe deployment readiness-demo -n artha-lab
```

### **Command 40:** `kubectl get endpointslices -n artha-lab -l kubernetes.io/service-name=readiness-demo -o yaml`

**Where:** HOST
**Why run it:** Inspect backend eligibility and readiness conditions.
**Decode every important part:** `-o yaml` exposes structured endpoint addresses and conditions.
**Expected observation:** EndpointSlices can show not-ready endpoints; routing should not treat them as ready.
**If the result differs:** If no slices, inspect Service/label selector and control plane.
**Safety / side effect:** Read only.

```
kubectl get endpointslices -n artha-lab -l kubernetes.io/service-name=readiness-demo -o yaml
```

### **Command 41:** `kubectl delete -f manifests/06-readiness-failure.yaml`

**Where:** HOST
**Why run it:** Clean up controlled failure.
**Decode every important part:** `delete -f` deletes all resources described in that file.
**Expected observation:** Resources terminate; deletion completes eventually.
**If the result differs:** Check finalizers if cleanup stalls.
**Safety / side effect:** DESTRUCTIVE to listed lab objects; do not use in production.

```
kubectl delete -f manifests/06-readiness-failure.yaml
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: health-demo
  namespace: artha-lab
spec:
  replicas: 1
  selector:
    matchLabels: {app: health-demo}
  template:
    metadata:
      labels: {app: health-demo}
    spec:
      terminationGracePeriodSeconds: 30
      containers:
      - name: web
        image: nginx:1.27
        readinessProbe:
          httpGet: {path: /, port: 80}
          periodSeconds: 5
          failureThreshold: 2
        livenessProbe:
          httpGet: {path: /, port: 80}
          periodSeconds: 10
          failureThreshold: 3
```

**Field-by-field explanation:**

- `terminationGracePeriodSeconds: 30`: kubelet gives shutdown grace before forced termination; the full ecosystem can take longer to stop forwarding.
- `readinessProbe.httpGet`: HTTP endpoint for traffic readiness.
- `periodSeconds: 5`: readiness check roughly every five seconds.

- `failureThreshold: 2`: consecutive failures before readiness becomes false.
- `livenessProbe`: only use a signal whose failure warrants restarting.
- `periodSeconds: 10` and `failureThreshold: 3`: liveness tolerance before restart.
- Both probes must match the app listening endpoint and port.
- Selector `app: health-demo` must match template labels.

**Change exercise:** Change readiness path to `/does-not-exist` in a disposable copy; predict Pod phase versus Ready condition.

## Common faults and diagnostic reasoning

- **Liveness kills an overloaded but recoverable Pod** — Avoid fragile dependencies in liveness; protect transient slow periods.
- **Readiness depends on shared outage** — All replicas might drop from endpoints simultaneously; design dependency checks intentionally.
- **PDB and node crash** — PDB only governs some voluntary disruptions, not unexpected node loss.

## Guided practice and evidence

1. Run the broken readiness Pod and compare Pod phase, READY column, Deployment Available replicas and EndpointSlice conditions.
2. Propose values for slow-starting Python app with 60-second startup; justify each timer.
3. Write a shutdown sequence from deletion to final SIGKILL and how to avoid dropped requests.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** How can a Pod be Running but an application Service have zero ready backends?

**Model answer:** The process can be running while readiness fails, so the Pod is excluded as a ready endpoint. A Service selector mismatch may also yield zero endpoints. Inspect probes, selector labels and EndpointSlices separately.

**Official references:** https://kubernetes.io/docs/concepts/workloads/pods/probes/, https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/

---

# Deep Lesson 09 — PersistentVolumes, Claims, StorageClasses, CSI and StatefulSets

**Goal:** Understand data ownership, binding, attach/mount and backup semantics.
**Starting point:** A local cluster; storage provisioning capability may be absent.
**Hinglish mental model:** Pod disposable hai, data lifecycle ko separately design karna padta hai.

## Teach the concept from zero

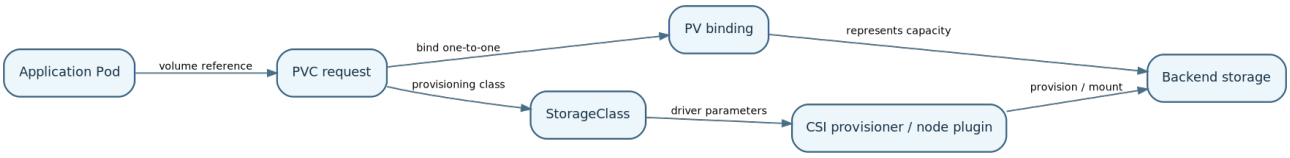
A container's writable filesystem is ephemeral. Kubernetes separates workload lifecycle from persistent data through **PersistentVolumeClaim (PVC)** and **PersistentVolume (PV)** resources. A PVC asks for capacity, access mode and possibly StorageClass. A PV represents provisioned storage and has a lifecycle independent of the Pod. StorageClasses describe provisioning parameters and reclaim behavior. Dynamic provisioning usually relies on a **CSI** driver. Claim-to-volume binding is controlled by API controllers and CSI components, not by a shell `mount` in your Deployment YAML.

A Pod refers to a PVC under `spec.volumes`, then a container attaches that Pod volume under `volumeMounts`. A PVC being Bound is necessary but does not guarantee that every node can mount it. Cloud block volumes may be tied to one availability zone and attach only to compatible nodes; mismatched topology causes scheduling or attach failures. Access modes such as `ReadWriteOnce` describe how a volume may be mounted; they do not make application data magically safe for concurrent writes. `ReadWriteMany` requires a driver/backend that actually supports it.

**StatefulSet** provides stable ordinal Pod identities and often stable persistent claims through volumeClaimTemplates. It is not a database HA solution: replication, backup, restore, consistency and promotion remain database-specific responsibilities. A PostgreSQL StatefulSet with one replica is not automatically highly available. Understand data and control-plane failure domains. For recovery, differentiate volume snapshot, database-consistent backup and point-in-time recovery (PITR). A snapshot taken during writes may require application/database-specific restoration steps.

If kind has no default StorageClass, a PVC can stay Pending. That is a legitimate educational outcome; don't claim an unverified storage system exists. The lab guide must check `kubectl get storageclass` first. Never delete a PVC holding needed data merely to clear an alert; reclaim policy may delete the underlying volume.

**Figure 9: PersistentVolumes, Claims, StorageClasses, CSI and StatefulSets**




**How to read this diagram, arrow by arrow:**

- **Application Pod → PVC request** (volume reference): Pod references a named claim from its namespace.
- **PVC request → StorageClass** (provisioning class): StorageClass selects dynamic provisioning behavior if available.
- **StorageClass → CSI provisioner / node plugin** (driver parameters): CSI controller/node components integrate external storage operations.
- **CSI provisioner / node plugin → Backend storage** (provision / mount): Actual backing store exists outside the Pod filesystem lifecycle.

- **PV binding → Backend storage** (represents capacity): PV captures Kubernetes representation of backend storage and reclaim policy.
- **PVC request → PV binding** (bind one-to-one): Claim binds to one compatible PV (or one dynamically created PV).

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 42: `kubectl get storageclass`

**Where:** HOST
**Why run it:** Check available provisioners before attempting a claim.
**Decode every important part:** `storageclass` is cluster-scoped; no namespace needed.
**Expected observation:** Names, provisioner and optional `(default)` annotation.
**If the result differs:** If none, guided PVC lab may remain Pending; document constraint.
**Safety / side effect:** Read only.

```
kubectl get storageclass
```

### Command 43: `kubectl apply -f manifests/07-pvc.yaml`

**Where:** HOST / lab pack
**Why run it:** Create a sample PVC and data-writer Pod.
**Decode every important part:** `apply` handles both YAML documents.
**Expected observation:** Claim might bind, then Pod starts if driver/volume works.
**If the result differs:** Inspect PVC Events, StorageClass and node topology if Pending.
**Safety / side effect:** Creates storage that can persist beyond Pod deletion; data lifecycle matters.

```
kubectl apply -f manifests/07-pvc.yaml
```

### Command 44: `kubectl get pvc demo-data -n artha-lab -w`

**Where:** HOST
**Why run it:** Observe binding or stuck Pending.
**Decode every important part:** `pvc` is short for persistentvolumeclaim; `-w` watches API changes.
**Expected observation:** Bound with capacity if provisioning works.
**If the result differs:** Read description/events for absent provisioner or restrictions.
**Safety / side effect:** Read only; Ctrl+C to stop.

```
kubectl get pvc demo-data -n artha-lab -w
```

### Command 45: `kubectl describe pvc demo-data -n artha-lab`

**Where:** HOST
**Why run it:** Diagnose provisioning failure with Events.
**Decode every important part:** `describe` includes StorageClass, bound PV name and event history.
**Expected observation:** Bound PV and/or useful provisioning Events.
**If the result differs:** Missing StorageClass, incompatible mode, quota and backend availability may matter.
**Safety / side effect:** Read only.

```
kubectl describe pvc demo-data -n artha-lab
```

### **Command 46:** `kubectl exec -n artha-lab data-writer -- cat /data/history.txt`

**Where:** HOST; only after Pod Running
**Why run it:** Prove the application Pod can read mounted data.
**Decode every important part:** `exec` executes in container; `--` separates kubectl flags from container command; `cat` reads a path inside.
**Expected observation:** Training text `first-write` or later persisted entries.
**If the result differs:** Check mount, PVC binding, container status and exact path.
**Safety / side effect:** Runs a read command inside Pod; never use unreviewed shell in production.

```
kubectl exec -n artha-lab data-writer -- cat /data/history.txt
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sample-data
  namespace: artha-lab
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
---
apiVersion: v1
kind: Pod
metadata:
  name: storage-reader
  namespace: artha-lab
spec:
  containers:
  - name: reader
    image: busybox:1.36
    command: ["sh", "-c", "ls /data; sleep 3600"]
    volumeMounts:
    - name: volume-data
      mountPath: /data
  volumes:
  - name: volume-data
    persistentVolumeClaim:
      claimName: sample-data
```

**Field-by-field explanation:**

- `PersistentVolumeClaim`: a namespaced request for storage.
- `accessModes: ReadWriteOnce`: compatible single-node mount semantics where supported; not a global file lock.
- `resources.requests.storage: 100Mi`: requested capacity in binary mebibytes.
- `Pod`: consumer of the PVC, with its own independent lifecycle.
- `volumeMounts[].name`: references Pod volume logical name.
- `mountPath: /data`: path visible inside a container.
- `volumes[].name`: Pod-level volume declaration referenced by mount.
- `persistentVolumeClaim.claimName`: must match PVC in same namespace.

**Change exercise:** Delete only the Pod, not the PVC, and explain which Kubernetes objects/data should remain.

## Common faults and diagnostic reasoning

- **PVC stays Pending** — Inspect default StorageClass, quota, provisioner logs and delayed binding policy.
- **PV Bound but Pod ContainerCreating** — Look for volume attach/mount events, CSI node plugin, zone and filesystem issues.
- **Claim deleted and data disappears** — Know StorageClass/PV reclaimPolicy and backup before deleting anything containing data.

## Guided practice and evidence

1. Write a diagram comparing Pod restart, PVC deletion and storage backend deletion.
2. If available, write a line to PVC-backed file, recreate Pod and verify persistence.
3. Design a database restore test and measure whether a snapshot alone meets RPO/RTO.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why does a StatefulSet not guarantee database data safety?

**Model answer:** Stable identity and claims support stateful workloads, but application replication, quorum, transaction consistency, backups, restore testing and disaster recovery are separate responsibilities.

**Official references:** https://kubernetes.io/docs/concepts/storage/persistent-volumes/, https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

---

# Deep Lesson 10 — Scheduler, requests, limits, QoS and autoscaling

**Goal:** Explain why a Pod stays Pending even if a node looks idle, and how HPA chooses replicas.
**Starting point:** You understand nodes, Pods and CPU/memory units.
**Hinglish mental model:** Request scheduling reservation hai; limit runtime boundary hai; HPA replicas decide karta hai.

## Teach the concept from zero

The scheduler does not place Pods by looking only at current `top` usage. It primarily accounts for declared **resource requests** and hard constraints when filtering nodes, then scores viable nodes. `requests.cpu: 250m` represents one-quarter of a CPU core for scheduling; `requests.memory: 128Mi` requests 128 mebibytes. `limits.cpu` may throttle CPU usage through cgroups; `limits.memory` can cause cgroup OOM kills. Requests that exceed node allocatable resources can cause Pending even when recent measured usage looks low. Overcommitting capacity is an engineering trade-off, not a free optimization.

Node selectors, required affinity, taints/tolerations, topology spread constraints and volume topology can veto scheduling. A toleration lets a Pod be eligible despite certain taints; it does not force scheduling onto a tainted node. A node affinity term is more expressive than a simple `nodeSelector`. Topology spread helps avoid concentrating replicas within one zone or node, but you need suitable actual topology labels and enough capacity.

**HPA** changes replica desired count of a scalable workload using a metric ratio and tolerances; CPU utilization-based HPA normally depends on CPU requests and the metrics pipeline. Simplified desired replicas ≈ current replicas × (observed utilization / target utilization), then bounded by min/max/stabilization behavior. **VPA** recommends/adjusts resource allocations, subject to mode and ecosystem support; **Cluster Autoscaler/Karpenter** addresses node capacity, not Pod replica count. Interactions can cause thrash if resource settings and workload signals are poorly chosen.

Three distinct questions: Will scheduler place a Pod? Will the container stay within its cgroup limits? Will autoscaling supply enough replicas and nodes fast enough? Answer them separately. Protect availability through testing and SLOs, not by setting enormous requests everywhere.

**Figure 10: Scheduler, requests, limits, QoS and autoscaling**




**How to read this diagram, arrow by arrow:**

- **Pending Pod → Scheduler filtering** (requirements): Requests, selectors, taints and topology constraints are considered.
- **Scheduler filtering → Scheduler scoring** (eligible nodes): Only nodes passing all required filters are scored.
- **Scheduler scoring → Chosen node** (bind Pod): Scheduler chooses/binds one eligible node through the API.
- **Metrics provider → HPA controller** (reported load): HPA reads metrics such as CPU utilization from a metrics provider.
- **HPA controller → Deployment replicas** (desired replicas): HPA updates desired replicas via scale subresource within configured bounds.
- **Deployment replicas → Pending Pod** (create Pods): Replicas controller creates/deletes Pods; scheduler still needs nodes.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 47: `kubectl get nodes -o wide`

**Where:** HOST
**Why run it:** Find available nodes and versions.
**Decode every important part:** Nodes are cluster-scoped; wide adds addresses.
**Expected observation:** At least kind control-plane node.
**If the result differs:** NotReady means investigate node condition.
**Safety / side effect:** Read only.

```
kubectl get nodes -o wide
```

### Command 48: `kubectl describe node kind-artha-lab-control-plane`

**Where:** HOST; replace with your actual node name
**Why run it:** Inspect capacity, allocatable and scheduled requested resources.
**Decode every important part:** `describe node NAME` prints resources, taints, conditions, non-terminated Pods.
**Expected observation:** Allocated resource percentages; capacity is not the same as allocatable.
**If the result differs:** If name differs, use output from `kubectl get nodes`.
**Safety / side effect:** Read only.

```
kubectl describe node kind-artha-lab-control-plane
```

### Command 49: `kubectl get pods -n artha-lab -o wide`

**Where:** HOST
**Why run it:** Correlate Pending status and selected node.
**Decode every important part:** `-o wide` includes NODE and IP.
**Expected observation:** Scheduled Pods show NODE; unscheduled may have blank NODE.
**If the result differs:** `describe` specific Pending Pod for FailedScheduling events.
**Safety / side effect:** Read only.

```
kubectl get pods -n artha-lab -o wide
```

### Command 50: `kubectl top pods -n artha-lab`

**Where:** HOST; metrics server installed
**Why run it:** Read current CPU/memory measured usage.
**Decode every important part:** `top pods` queries metrics API; namespaced with `-n`.
**Expected observation:** Per-Pod CPU and memory samples.
**If the result differs:** `Metrics API not available` means install supported metrics pipeline or skip; not necessarily an app fault.
**Safety / side effect:** Read only.

```
kubectl top pods -n artha-lab
```

### Command 51: `kubectl describe pod <POD-NAME> -n artha-lab`

**Where:** HOST; replace placeholder
**Why run it:** Read scheduler Events and resource settings.
**Decode every important part:** Angle-bracket placeholder is NOT literal shell syntax; replace with Pod name (without < >).
**Expected observation:** Event may say Insufficient cpu or unmatched selector.
**If the result differs:** Check `kubectl get pods` for correct name.
**Safety / side effect:** Read only.

```
kubectl describe pod <POD-NAME> -n artha-lab
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: v1
kind: Pod
metadata:
  name: scheduling-demo
  namespace: artha-lab
spec:
  containers:
  - name: app
    image: nginx:1.27
    resources:
      requests:
        cpu: 100m
        memory: 64Mi
      limits:
        cpu: 500m
        memory: 128Mi
```

**Field-by-field explanation:**

- `requests.cpu: 100m`: scheduling request for one-tenth of a core.
- `requests.memory: 64Mi`: scheduler considers this minimum allocation.
- `limits.cpu: 500m`: runtime CPU boundary/throttling behavior.
- `limits.memory: 128Mi`: cgroup memory limit and potential OOM kill.
- `resource units`: `m` is millicores; `Mi` is 2^20 bytes.
- `Pod` requests are considered together with all containers (plus relevant overhead), not only the first container.

**Change exercise:** Raise `requests.memory` beyond node allocatable in a disposable test and inspect FailedScheduling; do not oversubscribe your real laptop.

## Common faults and diagnostic reasoning

- **Pod Pending despite idle CPU** — Scheduling uses requested resources plus hard constraints, not instantaneous CPU usage alone.
- **CPU throttling** — Low latency can degrade if CPU limit too low; compare throttling metrics and performance.
- **HPA at 1 replica with unknown CPU** — CPU HPA needs meaningful requests and metrics; inspect conditions and metrics adapter.

## Guided practice and evidence

1. Calculate 250m × 4 Pods = 1 CPU of requests, before system overhead.
2. Force a Pending Pod in lab using impossible nodeSelector; diagnose its Event then remove.
3. Calculate desired HPA replicas for 3 Pods at 80% actual / 50% target, ignoring tolerance: roughly ceil(4.8)=5.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Can HPA solve a Pod Pending due to insufficient node memory?

**Model answer:** No. HPA changes workload replicas and can make capacity pressure worse. Node scaling or rightsizing/constraint changes are needed. HPA and node autoscaling solve different bottlenecks.

**Official references:** https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/, https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

---

# Deep Lesson 11 — Authentication, authorization, RBAC and Pod Security

**Goal:** Design least privilege and explain precisely who is allowed to perform which API verb.
**Starting point:** You know ServiceAccounts and namespaces.
**Hinglish mental model:** Authentication = tum kaun ho, authorization = tum kya kar sakte ho, admission = request policy pass karti hai?

## Teach the concept from zero

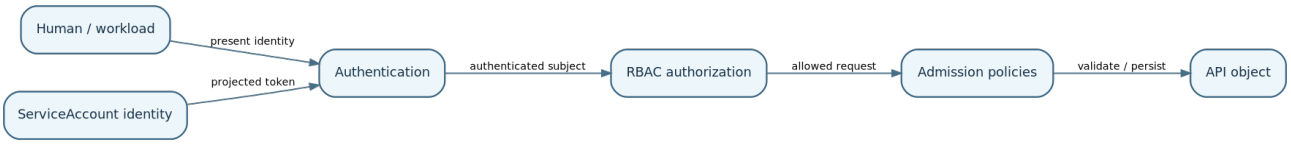
Kubernetes API security occurs in stages. **Authentication** determines identity (human user, ServiceAccount, trusted identity issuer). **Authorization** decides whether that identity may perform an action on a resource. **Admission** validates/mutates incoming API changes after prior checks. RBAC rules allow verbs (`get`, `list`, `watch`, `create`, `update`, `patch`, `delete`) on resource types in API groups. A **Role** is namespace-scoped; **ClusterRole** may be cluster-scoped or bound within a namespace with a RoleBinding. A **RoleBinding** attaches rules to subjects in its namespace. **ClusterRoleBinding** can grant cluster-wide permissions—use with care.

ServiceAccounts are workload identities inside Kubernetes, not human user records. Workloads may receive projected tokens depending on configuration; disable automatic token mounting if unneeded and prefer short-lived, appropriately scoped identity. Secrets readable by a Pod or ServiceAccount can be a major escalation route. `kubectl auth can-i` checks effective authorization for the specified identity; `--as` impersonation requires separate authority and should be used only in an authorized lab.

**Pod Security Admission (PSA)** applies the Privileged, Baseline and Restricted Pod Security Standards according to namespace labels. Restricted attempts to prevent many unsafe configurations, including escalation capabilities, and often requires `runAsNonRoot`, `allowPrivilegeEscalation: false`, a permitted seccomp profile and capability drops; exact requirements can evolve with versions. A securityContext does not replace image scanning, runtime detection, IAM segregation or NetworkPolicy. Also note that RBAC does not configure network traffic filtering: different control planes enforce different policies.

A useful security threat model asks: if an attacker compromises an app container, can they read API credentials? access Secrets? reach the node's metadata service? exec into other Pods? call cloud APIs? alter workloads? Design several independent boundaries. Least privilege is measured by denied actions as well as allowed ones.

**Figure 11: Authentication, authorization, RBAC and Pod Security**




**How to read this diagram, arrow by arrow:**

- **Human / workload → Authentication** (present identity): Credentials prove an identity; kubectl context alone is not proof of policy rights.
- **ServiceAccount identity → Authentication** (projected token): A workload uses its ServiceAccount credentials if configured and authorized.
- **Authentication → RBAC authorization** (authenticated subject): RBAC evaluates subject, verb, resource and scope.

- **RBAC authorization → Admission policies** (allowed request): Allowed requests still undergo admission validation and policy controls.
- **Admission policies → API object** (validate / persist): Object changes happen only if admission and schema validation pass.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 52: `kubectl apply -f manifests/08-rbac.yaml`

**Where:** HOST / lab pack
**Why run it:** Create a read-only ServiceAccount, Role and binding.
**Decode every important part:** Multi-document YAML applies three linked API objects.
**Expected observation:** view-pods, pod-reader and binding created.
**If the result differs:** Check subjects, `roleRef` and namespace if test fails.
**Safety / side effect:** Adds lab permissions only.

```
kubectl apply -f manifests/08-rbac.yaml
```

### Command 53: `kubectl auth can-i list pods -n artha-lab --as=system:serviceaccount:artha-lab:view-pods`

**Where:** HOST; requires impersonation permission
**Why run it:** Test a positive role capability.
**Decode every important part:** `auth can-i` checks access; `--as` sends impersonated username; serviceaccount identity has fixed prefix.
**Expected observation:** `yes` for listing Pods in artha-lab when RoleBinding matches.
**If the result differs:** If forbidden to impersonate, ask admin/test another authorized mechanism; do not bypass access controls.
**Safety / side effect:** Read-only authorization probe.

```
kubectl auth can-i list pods -n artha-lab --as=system:serviceaccount:artha-lab:view-pods
```

### Command 54: `kubectl auth can-i delete pods -n artha-lab --as=system:serviceaccount:artha-lab:view-pods`

**Where:** HOST; authorized impersonation
**Why run it:** Prove deletion is not granted to the read-only subject.
**Decode every important part:** Verb `delete` differs from `get`, `list`, `watch`.
**Expected observation:** `no` for the defined lab Role.
**If the result differs:** If yes, broader grants may exist—investigate bindings before assuming least privilege.
**Safety / side effect:** Read only.

```
kubectl auth can-i delete pods -n artha-lab --as=system:serviceaccount:artha-lab:view-pods
```

### **Command 55:** `kubectl apply -f manifests/12-security-restricted.yaml`

**Where:** HOST / disposable environment
**Why run it:** Create Restricted PSA namespace and compliant demonstration Pod.
**Decode every important part:** Namespace policy labels select PSA enforce mode; Pod must conform to requirements.
**Expected observation:** Namespace and Pod admitted if image/user/security requirements work.
**If the result differs:** If denied, inspect PSA warning and current policy requirements.
**Safety / side effect:** Creates restrictive namespace; isolate from main lab namespace.

```
kubectl apply -f manifests/12-security-restricted.yaml
```

### **Command 56:** `kubectl get role,rolebinding,serviceaccount -n artha-lab`

**Where:** HOST
**Why run it:** Inventory service identity and grants.
**Decode every important part:** Comma-separated resource kinds return a combined listing in namespace.
**Expected observation:** See pod-reader and binding.
**If the result differs:** Double-check whether a ClusterRoleBinding also grants more power.
**Safety / side effect:** Read only.

```
kubectl get role,rolebinding,serviceaccount -n artha-lab
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: reader
  namespace: artha-lab
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: reader-binding
  namespace: artha-lab
subjects:
- kind: ServiceAccount
  name: view-pods
  namespace: artha-lab
roleRef:
  kind: Role
  name: reader
  apiGroup: rbac.authorization.k8s.io
```

**Field-by-field explanation:**

- `Role`: RBAC rights within one namespace.
- `rules[]`: multiple additive rules possible.
- `apiGroups: [""]`: empty string means Kubernetes core API group, not wildcard.
- `resources`: matching API resources; `pods/log` is a subresource.
- `verbs`: only get/list/watch, not delete.

- `RoleBinding`: binds a Role/ClusterRole to one or more subjects.
- `subjects.kind`: ServiceAccount, a workload principal.
- `subjects.name` and namespace: must identify an existing ServiceAccount identity.
- `roleRef`: immutable binding reference to the Role; re-create binding to change roleRef.

**Change exercise:** Add `delete` as a hypothetical verb and explain how the blast radius changes; do not apply it to production.

## Common faults and diagnostic reasoning

- **SA appears denied despite Role** — Check RoleBinding subject namespace and roleRef, verb, apiGroup and resource.
- **Unexpected ability to delete** — A different binding may grant broader rights; RBAC allows are additive.
- **PSA Restricted breaks legacy workloads** — Audit and warn first in planned migrations; enable policy with compatibility testing.

## Guided practice and evidence

1. Create the least-privilege Role, test positive and negative permissions, and save results.
2. Write a threat model of a stolen ServiceAccount token and how to contain blast radius.
3. List restricted Pod securityContext fields and explain their runtime significance.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** What is the difference between RoleBinding and ClusterRoleBinding?

**Model answer:** RoleBinding grants the referenced Role/ClusterRole rules within one namespace. ClusterRoleBinding grants ClusterRole permissions cluster-wide; the underlying rule resource scope still matters. Use the narrowest necessary grant.

**Official references:** https://kubernetes.io/docs/reference/access-authn-authz/rbac/, https://kubernetes.io/docs/concepts/security/pod-security-standards/

---

# Deep Lesson 12 — CNI networking, NetworkPolicy and Linux dataplane

**Goal:** Explain veth pairs, Pod IPs, routes, conntrack and policy enforcement.
**Starting point:** You can distinguish Pod DNS, ClusterIP and Pod IP.
**Hinglish mental model:** Networking ko 3 parts me dekho: DNS resolution, packets ka routing, policy allow/deny.

## Teach the concept from zero

Kubernetes expects Pod networking to make Pod-to-Pod communication possible across nodes subject to policy. A **CNI** plugin configures interfaces and routes; implementations may use veth pairs, bridges, overlays, direct routing and eBPF programs. A Pod typically owns a network namespace and an interface. A veth pair joins the Pod namespace to host or virtual networking. A packet from Pod A destined for Pod B may traverse a local veth, routing path, overlay tunnel or external network, depending on deployment. The Kubernetes API does not dictate exactly which implementation is used.

**NetworkPolicy** is a namespaced API resource describing which traffic is allowed for selected Pods. It works only if your CNI actually enforces it. Policies are additive: an allow rule from another matching policy may make traffic possible even when a new restrictive policy is present; a **default deny** selects Pods and defines zero allowed rules for a direction. Ingress and egress are independently controlled. DNS egress often needs explicit allowance after default deny, including traffic to your actual cluster DNS Pods/ports. Policies use selectors, ports and IPBlocks with important caveats around NAT; do not assume a single CNI-independent traffic path.

Linux `conntrack` tracks network flows for connection-oriented NAT/firewall behavior, but packet forwarding can be implemented differently with eBPF, and implementations vary. High connection counts, NAT state pressure or misapplied policies can cause intermittent failures. Use `ip addr`, `ip route`, `ss`, `getent`, and authorized packet captures to locate the broken segment. Be cautious on an 8 GB laptop and never use `nft flush ruleset` or arbitrary firewall deletion as a diagnostic in shared environments.

Think in **four failure categories**: name resolution; network reachability; policy/authorization; application protocol. A Service returning HTTP 500 is reachable at TCP/HTTP, but the application is failing. A connection timeout is different. Make this separation explicit in incident reports.

**Figure 12: CNI networking, NetworkPolicy and Linux dataplane**




**How to read this diagram, arrow by arrow:**

- **Pod A netns → veth / host net** (packet out): Pod interface sends packet through its network namespace boundary.
- **veth / host net → CNI routing / overlay** (forward): CNI-specific configuration provides routing or tunneling across hosts.
- **CNI routing / overlay → NetworkPolicy enforcement** (enforce if capable): Policy handling depends on CNI and dataplane; exact order relative to NAT varies.
- **NetworkPolicy enforcement → Pod B netns** (deliver allowed): Allowed traffic can reach the backend Pod namespace/port.

- **Service dataplane → CNI routing / overlay** (service backend mapping): Service virtual IP traffic is translated or redirected toward backend routes.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 57: `kubectl get pods -n kube-system -o wide`

**Where:** HOST
**Why run it:** Identify networking/system Pods and node placement.
**Decode every important part:** `-n kube-system` targets common system namespace; `-o wide` includes node/IP.
**Expected observation:** CNI and DNS workloads may be visible; control-plane architecture varies.
**If the result differs:** Some managed CNI deployments use host services or different namespaces.
**Safety / side effect:** Read only.

```
kubectl get pods -n kube-system -o wide
```

### Command 58: `kubectl get networkpolicy -n artha-lab`

**Where:** HOST
**Why run it:** Inventory declared namespace network rules.
**Decode every important part:** `networkpolicy` is namespaced API kind; no implicit interpretation of enforcement.
**Expected observation:** Names of policies or no resources.
**If the result differs:** No policy does not prove unrestricted networking if external controls exist.
**Safety / side effect:** Read only.

```
kubectl get networkpolicy -n artha-lab
```

### Command 59: `kubectl apply -f manifests/11-default-deny-policy.yaml`

**Where:** HOST / disposable lab only
**Why run it:** Demonstrate zero-allow ingress/egress intent.
**Decode every important part:** Policy selects all namespace Pods via `podSelector: {}` and both policyTypes.
**Expected observation:** Policy object created; traffic impact ONLY if local CNI enforces policy.
**If the result differs:** Do not infer enforcement from successful `apply`; test traffic.
**Safety / side effect:** Can break DNS/traffic in lab; never apply to shared namespaces.

```
kubectl apply -f manifests/11-default-deny-policy.yaml
```

### Command 60: `kubectl describe networkpolicy demo-default-deny -n artha-lab`

**Where:** HOST
**Why run it:** Inspect selected Pods/directions/rules.
**Decode every important part:** `describe` prints policy selectors and rule summaries.
**Expected observation:** Ingress/egress isolation declared with no explicit allow rules.
**If the result differs:** The API accepts policies even where CNI enforcement is absent.
**Safety / side effect:** Read only.

```
kubectl describe networkpolicy demo-default-deny -n artha-lab
```

### Command 61: `ip route show`

**Where:** HOST or authorized NODE
**Why run it:** Understand host-side network routes.
**Decode every important part:** `ip` is iproute2; `route show` lists routing decisions.
**Expected observation:** Routes with device names and prefixes.
**If the result differs:** Inside container namespace output differs from host.
**Safety / side effect:** Read only; do not alter routes.

```
ip route show
```

### Command 62: `ss -ntp`

**Where:** Authorized NODE/host
**Why run it:** See established TCP sockets and owning processes when permitted.
**Decode every important part:** `ss` socket summary; `-n` numeric, `-t` TCP, `-p` processes.
**Expected observation:** TCP connection details; process columns may require privilege.
**If the result differs:** Lack of privileges can hide process info.
**Safety / side effect:** Read only.

```
ss -ntp
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ingress-only-web
  namespace: artha-lab
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 80
```

**Field-by-field explanation:**

- `kind: NetworkPolicy`: declared network allow policy; needs implementing CNI.
- `spec.podSelector`: selects **destination Pods** protected by this policy.
- `app: web`: exact label match.
- `policyTypes: Ingress`: ingress isolation rules; egress unaffected by this policy alone.
- `ingress[]`: each item is an allowed inbound rule.
- `from[].podSelector`: matches source Pods in same namespace by default.
- `ports[].protocol: TCP`: only TCP packets on allowed target port.
- `port: 80`: inbound destination port on selected Pods.

**Change exercise:** Switch source selector to a non-existent label and explain the expected traffic if policy is enforced and no other allowing policy exists.

## Common faults and diagnostic reasoning

- **Default deny causes DNS failures** — Allow needed DNS egress intentionally and verify selector semantics.
- **Policy applied but packets flow** — CNI enforcement capability may be missing or another policy allows traffic.
- **Pod reachable by IP but not hostname** — Investigate DNS and Service discovery before rewriting routes.

## Guided practice and evidence

1. Draw DNS vs dataplane vs policy planes and classify five failure symptoms.
2. Use the original default-deny lab on a disposable namespace, test whether it actually enforces, and document the CNI.
3. Explain a packet from frontend Pod to backend Pod on another node, including CNI-dependent branches.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** If a NetworkPolicy object exists, does that guarantee traffic is blocked?

**Model answer:** No. Enforcement depends on a compatible CNI and interactions among matching additive policies, direction and selectors. A successful API create is not a dataplane validation.

**Official references:** https://kubernetes.io/docs/concepts/services-networking/network-policies/, https://kubernetes.io/docs/concepts/cluster-administration/networking/

---

# Deep Lesson 13 — Control-plane HA, etcd quorum, backups and upgrades

**Goal:** Reason about failure domains, disaster recovery and safe upgrade procedure without risking production.
**Starting point:** You have read Kubernetes architecture and learned to distinguish workload and API state.
**Hinglish mental model:** HA ≠ backup; etcd quorum ≠ app data recovery; upgrade ke pehle restore test zaroori.

## Teach the concept from zero

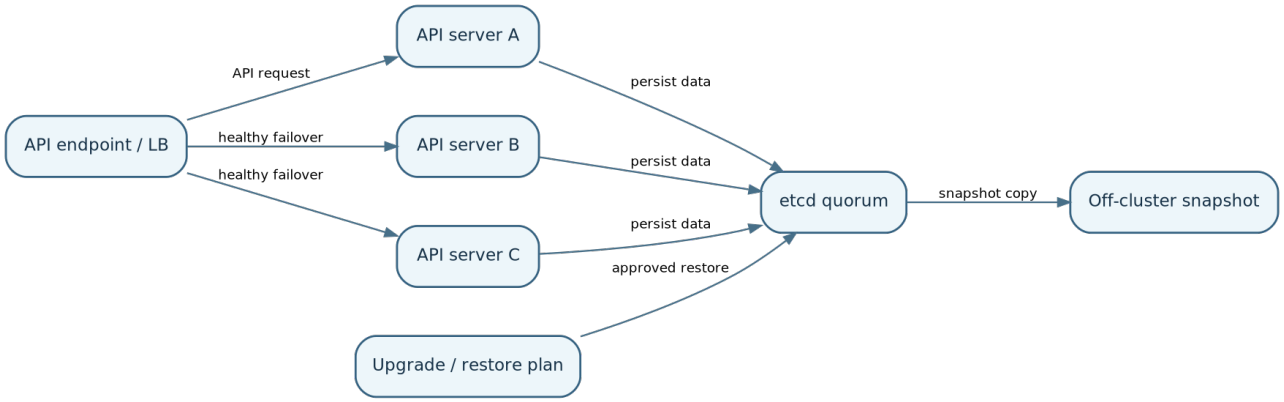
A self-managed highly available control plane generally needs multiple API server instances behind a stable endpoint, controller-manager/scheduler leader election, certificate and network consistency, and highly available **etcd**. etcd uses a consensus protocol and typically requires a majority (quorum) of members to commit writes: a 3-member set can lose 1, a 5-member set can lose 2; a 2-member set can lose **zero** without losing quorum. More members are not always better due to write/latency costs.

**etcd stores Kubernetes API data**, not your application's PVC database contents. Protect it with restricted network access, TLS, backups and restore drills. **RPO** = maximum acceptable data loss time; **RTO** = time target to restore service. Snapshot creation is not DR completion: verify integrity, store away from the cluster, protect encryption keys and actually rehearse restore in a safe environment. A multi-node etcd restore is a coordinated cluster recovery operation and should never be improvised from a copy/pasted command.

Upgrades require checking **supported version skew** between API servers, kubelet, kubectl, CNI, CSI and other add-ons. Take backups, validate cluster health, review deprecated API usage, select a canary/failure domain, and plan node cordon/drain with PDB implications. `kubectl drain` requests evictions and can disrupt workloads; don't use on a shared node casually. A rollback of binaries may not reverse etcd state/API migrations; the documented recovery route depends on your version and distribution.

On an 8 GB Debian laptop, it is valuable to **design and simulate** HA scenarios on paper, even when you cannot afford realistic 3-control-plane testing. State which parts are hypothetical. For staff-level interviews explain the failure timeline, whether existing workloads keep running, how traffic fails over, who has authority to perform restores, and objective go/no-go gates.

**Figure 13: Control-plane HA, etcd quorum, backups and upgrades**




**How to read this diagram, arrow by arrow:**

- **API endpoint / LB → API server A** (API request): Clients access one stable API endpoint; LB selects a healthy server.
- **API endpoint / LB → API server B** (healthy failover): A second API server keeps API requests available after one instance failure.
- **API endpoint / LB → API server C** (healthy failover): Third server improves fault tolerance if components sit in independent failure domains.
- **API server A → etcd quorum** (persist data): API servers use etcd as backing data store.
- **API server B → etcd quorum** (persist data): All API instances must see compatible cluster state.
- **API server C → etcd quorum** (persist data): Persistent writes require healthy storage/quorum.
- **etcd quorum → Off-cluster snapshot** (snapshot copy): Offline snapshot gives a possible restore point, not live HA.
- **Upgrade / restore plan → etcd quorum** (approved restore): Restoration requires an explicit coordinated runbook and appropriate authority.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 63: `kubectl version --client`

**Where:** HOST
**Why run it:** Check your local kubectl version before compatibility decisions.
**Decode every important part:** `--client` avoids server inspection and prints client build.
**Expected observation:** Client version and tool metadata.
**If the result differs:** If server is unavailable this command can still work.
**Safety / side effect:** Read only.

```
kubectl version --client
```

### Command 64: `kubectl get nodes -o wide`

**Where:** HOST
**Why run it:** Record node kubelet versions.
**Decode every important part:** Node `VERSION` column commonly reflects kubelet version.
**Expected observation:** Versions shown for each node.
**If the result differs:** Version mismatch needs evaluation against skew policy, not immediate deletion.
**Safety / side effect:** Read only.

```
kubectl get nodes -o wide
```

### Command 65: `kubectl get --raw /readyz`

**Where:** HOST
**Why run it:** Check API server readiness before planned maintenance.
**Decode every important part:** `--raw` sends API endpoint request.
**Expected observation:** `ok` when ready, if endpoint accessible.
**If the result differs:** Do not confuse API readiness with multi-AZ availability or app uptime.
**Safety / side effect:** Read only.

```
kubectl get --raw /readyz
```

### Command 66: `kubectl get pdb -A`

**Where:** HOST
**Why run it:** Find PodDisruptionBudgets that influence evictions.
**Decode every important part:** `pdb` abbreviates policy PodDisruptionBudget; `-A` all namespaces.
**Expected observation:** Minimum availability settings where configured.
**If the result differs:** PDB absence may allow voluntary eviction of all Pods; PDB presence can block drain.
**Safety / side effect:** Read only.

```
kubectl get pdb -A
```

### Command 67: `kubectl get events -A --sort-by=.lastTimestamp`

**Where:** HOST
**Why run it:** Inspect cross-namespace recent events before maintenance.
**Decode every important part:** `-A` all namespaces; `--sort-by` sorts by field name.
**Expected observation:** Chronological Events to inform state.
**If the result differs:** Events can expire; do not depend on them as sole audit/monitor history.
**Safety / side effect:** Read only.

```
kubectl get events -A --sort-by=.lastTimestamp
```

## Common faults and diagnostic reasoning

- **Two-member etcd cluster** — Majority of 2 is 2, so one failure loses quorum; choose deliberate 3 or 5 where needed.
- **Backups exist but cannot restore** — Run rehearsals and record restore times, secrets/certs handling and integrity.
- **Drain blocked by PDB** — Capacity and availability must be adjusted deliberately; do not force evictions blindly.

## Guided practice and evidence

1. Compute majority and tolerated failures for 1, 2, 3, 4 and 5 etcd members.
2. Write an HA diagram for three control-plane nodes in different zones; list what remains a single point of failure.
3. Prepare an authorized upgrade runbook covering backups, deprecated APIs, CNI/CSI, canary, monitoring and stop conditions.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** If etcd quorum is lost, do running Pods instantly stop?

**Model answer:** No. Existing containers may keep running, but API writes and controller reconciliation can fail; node and workload behavior varies. Restore API state safely before claiming control-plane recovery, and separately validate application traffic and data.

**Official references:**
https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/,
https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/,
https://kubernetes.io/releases/version-skew-policy/

# Deep Lesson 14 — Helm, Kustomize, declarative releases and GitOps

**Goal:** Package manifests safely and distinguish rendering, reconciliation and deployment success.
**Starting point:** You have hand-written Deployments, ConfigMaps and Services.
**Hinglish mental model:** Helm templates manifest banata hai; GitOps controller desired Git state ke saath cluster ko sync karta hai.

## Teach the concept from zero

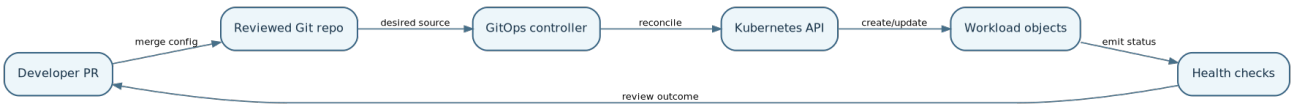
Raw YAML is an excellent learning foundation, but large apps have repeated values for image, replicas, namespace, resource limits and hostname. **Helm** packages Kubernetes resources as **charts**. Templates are rendered from values into YAML objects, then installed/upgraded as a release; `helm template` displays render output without changing a cluster. **Kustomize** uses overlays and patches instead of a template language, enabling base + environment variations. Neither eliminates the need to understand emitted Kubernetes API resources.

GitOps treats Git as a reviewed desired-state source. A controller such as Argo CD detects changes between a declared source and the cluster and can synchronize according to policy. One important difference: a deployment tool executed by a CI worker is a push action; a pull-based GitOps controller acts from inside its own operating boundary. That can reduce broad CI credentials, but it shifts trust to Git, controller service accounts and admission policies. GitOps cannot guarantee that application-level migration or feature behavior is correct.

Prefer immutable image digests for promotion, reproducible chart versions, peer-reviewed PRs, protected branches, policy/manifest validation, rollout checks, and environment-specific approval gates. `helm upgrade --install` changes cluster state; first inspect `helm template` and diff. The highest-value exercise is not memorizing chart syntax: it is answering **who can change production, how is that change reviewed, how is a failed release detected, and how is it rolled back?**

Reconciliation conflicts arise when people manually patch resources GitOps also owns. Decide whether self-healing should revert drift automatically, when exceptions are valid, and how to communicate them. A drift alert is evidence about declared configuration, not proof of a compromised cluster.

**Figure 14: Helm, Kustomize, declarative releases and GitOps**




**How to read this diagram, arrow by arrow:**

- **Developer PR → Reviewed Git repo** (merge config): Human change receives version-control review and approval.
- **Reviewed Git repo → GitOps controller** (desired source): Reconciler reads selected revision/path and target environment.
- **GitOps controller → Kubernetes API** (reconcile): Controller submits authorized API changes when policy permits.
- **Kubernetes API → Workload objects** (create/update): Standard Kubernetes controllers create/modify actual runtime resources.
- **Workload objects → Health checks** (emit status): Readiness, logs, metrics and tests provide operational feedback.

- **Health checks → Developer PR** (review outcome): Review gates assess actual quality; green sync status alone is insufficient.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 68: `helm version --short`

**Where:** HOST; Helm installed
**Why run it:** Check Helm availability before chart exercises.
**Decode every important part:** `helm version` reports client, `--short` condenses output.
**Expected observation:** A version string.
**If the result differs:** If absent, follow official Helm installation instructions with verified binary.
**Safety / side effect:** Read only.

```
helm version --short
```

### Command 69: `helm create demo-chart`

**Where:** HOST inside lab folder
**Why run it:** Generate a chart scaffold for learning.
**Decode every important part:** `create` generates templates, chart metadata and default values in a directory.
**Expected observation:** New local `demo-chart/` tree.
**If the result differs:** Refuse/rename if target directory already contains files.
**Safety / side effect:** Creates local files; review before editing.

```
helm create demo-chart
```

### Command 70: `helm lint ./demo-chart`

**Where:** HOST
**Why run it:** Catch chart structure/template issues early.
**Decode every important part:** `lint` inspects a local chart and reports issues.
**Expected observation:** Lint summary with 0 failures for valid scaffold.
**If the result differs:** Fix reported YAML/template errors before trying install.
**Safety / side effect:** Read-only to cluster; reads local files.

```
helm lint ./demo-chart
```

### Command 71: `helm template web ./demo-chart`

**Where:** HOST
**Why run it:** See rendered Kubernetes YAML without installing.
**Decode every important part:** `template` renders chart locally; `web` is release name, directory identifies chart.
**Expected observation:** One or more YAML manifests to inspect field-by-field.
**If the result differs:** Errors point to values/templates; rendered YAML can be valid syntactically but unsafe semantically.
**Safety / side effect:** No cluster mutation.

```
helm template web ./demo-chart
```

### Command 72: `kubectl kustomize ./overlays/dev`

**Where:** HOST; an overlay directory that you create
**Why run it:** Render a Kustomize overlay to inspect final YAML.
**Decode every important part:** `kustomize` renders files from `kustomization.yaml`; path selects overlay.
**Expected observation:** Rendered YAML output.
**If the result differs:** If no overlay directory exists, create base/overlays first; command is not plug-and-play.
**Safety / side effect:** No cluster mutation.

```
kubectl kustomize ./overlays/dev
```

## Common faults and diagnostic reasoning

- **Rendered YAML valid but app fails** — Render checks only syntax/objects; workload runtime and data migrations still need testing.
- **Manual hotfix overwritten** — GitOps reconciliation may revert drift; update source-of-truth via approved process.
- **Secret committed to values.yaml** — Use a secret integration and credential scanning; do not store raw production secrets in charts.

## Guided practice and evidence

1. Create a Helm chart and identify exactly where image and replica values end up in rendered Deployment.
2. Design dev and prod Kustomize overlays with different replicas and request sizes.
3. Write a GitOps production PR checklist including smoke tests, SLO gates, rollback and approvals.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** What is a key difference between Helm and Argo CD?

**Model answer:** Helm renders/packages/installs Kubernetes resources as chart releases; Argo CD continuously compares configured source with target cluster state and reconciles drift. They are frequently used together.

**Official references:** https://helm.sh/docs/, https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/, https://argo-cd.readthedocs.io/en/stable/

---

# Deep Lesson 15 — Amazon EKS, VPC, IAM and Terraform from first principles

**Goal:** Design a managed Kubernetes platform and know exactly which responsibilities remain yours.
**Starting point:** Kubernetes internals, basic AWS VPC, IAM, subnets and route tables.
**Hinglish mental model:** EKS control plane AWS chalata hai, lekin security, workload, network aur cost ki responsibility khatam nahi hoti.

## Teach the concept from zero

**Amazon EKS** is a managed Kubernetes service. AWS operates the managed Kubernetes control plane, but customers still design VPC networking, cluster access, node capacity, pod identity, application RBAC, storage, observability and workload security. An EKS cluster endpoint can be public, private or configured for appropriate access patterns. Worker nodes typically join inside selected subnets; private node subnets require planned routes and/or VPC endpoints for registries, STS, EKS and other AWS services that workloads need. A private subnet does not mean traffic is secure or services are magically reachable.

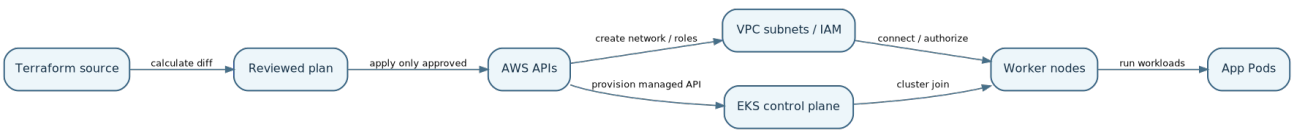
Networking has several layers: route tables move packets; security groups are stateful network filters attached to resources/ENIs; NACLs are stateless subnet-layer filters; the AWS VPC CNI controls Pod IP allocation according to its configuration; the Kubernetes Service dataplane routes Service destinations; ingress controllers / load balancers publish selected traffic. Debug the failing layer rather than treating all connectivity issues as DNS.

**IAM** controls AWS API actions; **Kubernetes RBAC** controls Kubernetes API actions. For workloads needing AWS resources, use approved workload identity mechanisms such as EKS Pod Identity or IAM Roles for Service Accounts (IRSA), according to actual supported scenarios and provider requirements. Do not mount long-lived AWS access keys in Pods as the default choice. EKS access entries and Kubernetes policy are related but not interchangeable. Carefully distinguish the cluster creator/admin, node role, add-on role and each workload role.

**Terraform** expresses infrastructure desired state with providers, resources, variables, modules and state. `terraform plan` computes proposed changes; **it can involve cloud API reads** and provider installation even though it does not itself apply changes. `terraform apply` performs mutation and can generate significant charges. Terraform state can contain secrets: use properly secured remote backend, versioning, lock support and least-privilege access. Avoid relying on old recipes whose state-lock mechanisms are deprecated or have changed. A plan preview is not an authorization to perform external actions.

For an 8 GB zero-budget lab, start with diagrams, `terraform fmt`, no-cloud HCL parsing, and reviewed architecture. EKS is an optional authorized cloud lab and can incur resource, load-balancer, NAT gateway and traffic costs quickly. Any company/founder system should require explicit approval before network calls and paid provisioning.

**Figure 15: Amazon EKS, VPC, IAM and Terraform from first principles**




**How to read this diagram, arrow by arrow:**

- **Terraform source → Reviewed plan** (calculate diff): Terraform reads code and state to propose adds, updates and destroys.

- **Reviewed plan → AWS APIs** (apply only approved): Only a separately approved apply makes API mutations; even plan may read APIs.
- **AWS APIs → VPC subnets / IAM** (create network / roles): Networking and identity resources must exist and be correctly scoped.
- **AWS APIs → EKS control plane** (provision managed API): AWS manages control plane lifecycle within the managed service scope.
- **VPC subnets / IAM → Worker nodes** (connect / authorize): Worker nodes require valid subnets, routes, IAM roles and security rules.
- **EKS control plane → Worker nodes** (cluster join): Nodes register with the cluster and are authorized to operate.
- **Worker nodes → App Pods** (run workloads): Pods are scheduled to managed/self-managed/Fargate capacity as configured.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 73: `terraform version`

**Where:** HOST; Terraform installed
**Why run it:** Check Terraform version before HCL examples.
**Decode every important part:** `terraform version` prints installed CLI and provider context when present.
**Expected observation:** A version string.
**If the result differs:** Follow official HashiCorp docs if absent; version pin reproducible projects.
**Safety / side effect:** Read only.

```
terraform version
```

### Command 74: `terraform fmt -check -recursive`

**Where:** HOST in an existing Terraform project
**Why run it:** Check style across HCL files.
**Decode every important part:** `fmt` formats; `-check` exits nonzero rather than rewriting; `-recursive` visits children.
**Expected observation:** Success when all `.tf` files are formatted.
**If the result differs:** If no HCL files, command provides little learning; create a local module first.
**Safety / side effect:** No cloud mutation; local read-only style check.

```
terraform fmt -check -recursive
```

### Command 75: `terraform validate`

**Where:** HOST in a fully initialized Terraform directory
**Why run it:** Validate Terraform configuration consistency.
**Decode every important part:** `validate` checks config syntax/references; providers/modules may require prior `init`.
**Expected observation:** Success message if configuration valid.
**If the result differs:** If provider not installed, `terraform init` may need network permission; do not run unknowingly.
**Safety / side effect:** No cloud apply; local validation (with installed provider schema).

```
terraform validate
```

### Command 76: `kubectl config current-context`

**Where:** HOST
**Why run it:** Check Kubernetes context before EKS operations.
**Decode every important part:** Print active kubeconfig context; EKS contexts may coexist with kind.
**Expected observation:** Current context name.
**If the result differs:** Stop if unexpected; switch only deliberately.
**Safety / side effect:** Read only.

```
kubectl config current-context
```

### Command 77: `aws sts get-caller-identity`

**Where:** HOST; only in an approved AWS environment
**Why run it:** Confirm AWS account and principal before any costly action.
**Decode every important part:** `aws sts` invokes Security Token Service; `get-caller-identity` returns principal/account.
**Expected observation:** Account and ARN for authorized identity.
**If the result differs:** Credential/network errors need authorized remediation, not bypass.
**Safety / side effect:** NETWORK CALL to AWS; do not run without explicit permission.

```
aws sts get-caller-identity
```

## Common faults and diagnostic reasoning

- **EKS private endpoint unreachable** — Check VPN/Direct Connect/bastion routing, endpoint access and DNS before changing IAM.
- **Pod can list Kubernetes objects but not S3** — Kubernetes RBAC and AWS IAM are separate policy systems.
- **Terraform plan has deletes** — Treat as a stop/go gate; investigate state drift, module upgrades and resource address changes.
- **Unexpected cloud invoice** — Estimate LB, NAT, nodes, EKS fees, disks and traffic before any provision; tear down with evidence.

## Guided practice and evidence

1. Create an EKS architecture drawing with private nodes, route tables, security groups, IAM and workload identities.
2. Write a table mapping EKS responsibilities between AWS and customer.
3. Author a read-only plan review checklist explicitly prohibiting `terraform apply` without approval.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** If AWS operates EKS control plane, are you relieved of cluster security responsibility?

**Model answer:** No. AWS manages the service-side control-plane scope, while customers still own workload identity, access, Pod security, network policies, configuration, node/add-on choices, data and monitoring under the shared-responsibility model.

**Official references:** https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html, https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html,

https://developer.hashicorp.com/terraform/docs

# Deep Lesson 16 — CI/CD, image provenance, SBOMs and deployment safety

**Goal:** Build a trustworthy path from code change to measured deployment.
**Starting point:** You can read YAML and understand a GitOps controller.
**Hinglish mental model:** Pipeline green hona delivery ka proof hai, production business behavior ka proof nahi.

## Teach the concept from zero

A modern release system separates **build**, **test**, **scan**, **sign/attest**, **publish**, **promote**, **deploy** and **verify**. The goal is to know which source commit, dependencies and build environment produced the exact artifact now running. Image tags are convenient but can move; digest-pinning improves immutability, and signature/provenance checks can prove claims about origin when keys and policy are managed correctly. **SBOM** inventories software components and supports vulnerability analysis; an SBOM is not a guarantee that vulnerabilities are absent.

In Kubernetes pipelines, a CI job might run unit/integration tests, build an OCI image, scan it with an approved tool, generate SBOM, push to a registry, and update a GitOps manifest to a digest. A GitOps reconciler or deployment orchestrator then changes the cluster. Separating CI credentials from broad production write access reduces blast radius. PR reviews and approvals gate promotion. Avoid handing Jenkins a cluster-admin kubeconfig when a narrower service role will suffice.

Deployment health should combine Kubernetes rollout readiness with HTTP smoke tests, synthetic transactions, error rates, tail latency, saturation and business invariants. A 100% passing unit-test stage does not guarantee database backward compatibility. A rollback may fix app code but not reverse a destructive schema migration, so design expand/contract migrations. Keep an auditable record of who approved change and which checks passed.

Threat model the pipeline: compromised source control, malicious dependencies, tampered runner, leaked registry token, mutable image tag, overbroad IAM, and compromised admission webhook. Defense-in-depth can include pinned actions/dependencies, secret scanning, reproducible builds where feasible, separated trust domains, artifact signing and admission verification. Security controls should be tested with safe dummy artifacts rather than production credential exposure.

**Figure 16: CI/CD, image provenance, SBOMs and deployment safety**




**How to read this diagram, arrow by arrow:**

- **Signed / reviewed commit → CI runner** (trigger): Approved commits trigger controlled build environments.
- **CI runner → Tests + security checks** (build / inspect): Pipeline performs tests, security scanning, SBOM and attestation where configured.
- **Tests + security checks → Artifact registry** (publish digest): Artifact is published with immutable digest reference.
- **Artifact registry → Promotion GitOps repo** (pin artifact): Deployment source selects a particular reviewed artifact.
- **Promotion GitOps repo → Kubernetes rollout** (sync desired version): Deployment applies desired state via authorized controller.

- **Kubernetes rollout → Runtime SLO verification** (observe behavior): Postdeploy monitoring validates user-level reliability beyond Pod readiness.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 78: `git rev-parse HEAD`

**Where:** HOST within a Git repository
**Why run it:** Record exact commit SHA for build evidence.
**Decode every important part:** `git rev-parse` resolves repository references; `HEAD` is checked-out commit.
**Expected observation:** A commit hash.
**If the result differs:** If not a repository, create one or change to correct project path.
**Safety / side effect:** Read only.

```
git rev-parse HEAD
```

### Command 79: `git status --short`

**Where:** HOST within a Git repository
**Why run it:** Check for uncommitted changes before building.
**Decode every important part:** `status` inspects work tree; `--short` makes compact status.
**Expected observation:** Empty when clean, or modified/untracked entries.
**If the result differs:** Untracked config may affect local builds but not CI artifact source.
**Safety / side effect:** Read only.

```
git status --short
```

### Command 80: `kubectl rollout history deployment/web -n artha-lab`

**Where:** HOST
**Why run it:** Inspect Deployment revision history after release.
**Decode every important part:** `rollout history` lists revisions saved for template changes.
**Expected observation:** Revision identifiers; details depend on change-cause annotation/tooling.
**If the result differs:** History is not a substitute for Git commit and image digest provenance.
**Safety / side effect:** Read only.

```
kubectl rollout history deployment/web -n artha-lab
```

### Command 81: `kubectl rollout status deployment/web -n artha-lab --timeout=180s`

**Where:** HOST
**Why run it:** Wait for infrastructure-level rollout convergence.
**Decode every important part:** Bounded wait, not an application SLO test.
**Expected observation:** Successful rollout message if available replicas meet conditions.
**If the result differs:** Investigate status/Events rather than ignoring timeouts.
**Safety / side effect:** Read only/wait.

```
kubectl rollout status deployment/web -n artha-lab --timeout=180s
```

**Command 82:** `kubectl get deployment web -n artha-lab -o jsonpath="{.spec.template.spec.containers[0].image}"`

**Where:** HOST
**Why run it:** Record the image actually declared in Deployment template.
**Decode every important part:** JSONPath traverses spec, selects first container `[0]`, prints image reference.
**Expected observation:** A tag or digest reference.
**If the result differs:** If multiple containers, inspect each; a tag alone is mutable.
**Safety / side effect:** Read only.

```
kubectl get deployment web -n artha-lab -o jsonpath="{.spec.template.spec.containers[0].image}"
```

## Common faults and diagnostic reasoning

- **Pipeline passes but prod errors** — Add application-level contract/latency tests and real SLO gates.
- **Tag moved between stages** — Prefer promotion of the same immutable image digest.
- **Runner holds powerful kubeconfig** — Restrict CI authority; use GitOps pull workflows and narrowly scoped tokens where possible.

## Guided practice and evidence

1. Draw the commit-to-Pod supply chain with exact evidence stored at each hop.
2. Write a mock CI pipeline plan with build, test, scan, SBOM, digest-promotion, approval, deploy, smoke and rollback.
3. Demonstrate how to identify a running image reference, and explain the distinction between tag and digest.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why are SBOM and signature verification different?

**Model answer:** An SBOM describes dependency composition; signatures/attestations bind origin and build claims to an artifact under a trust model. One does not replace the other, and both still require vulnerability response and secure execution.

**Official references:** https://kubernetes.io/docs/concepts/containers/images/, https://slsa.dev/spec/, https://docs.sigstore.dev/

---

# Deep Lesson 17 — Monitoring, logs, traces and incident diagnosis

**Goal:** Diagnose an incident using metrics, events, logs and request traces while protecting reliability.
**Starting point:** Basic Linux and Pod/Service understanding.
**Hinglish mental model:** Metrics batate hain kitna bura; logs batate hain kya hua; trace batata hai request kahan gayi.

## Teach the concept from zero

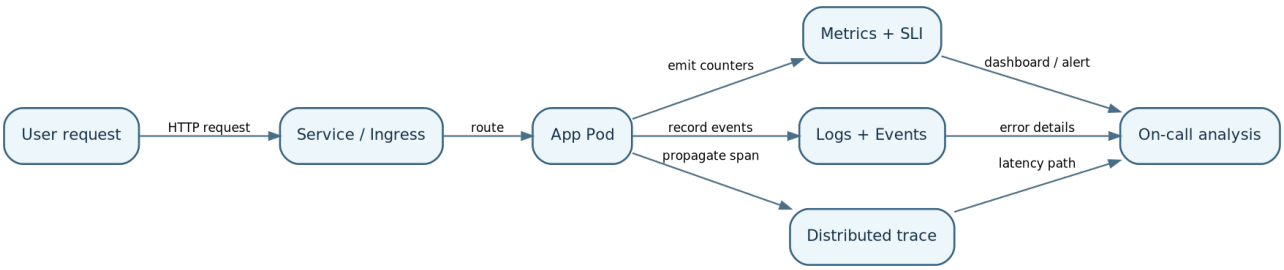
Operational evidence includes **metrics** (numeric time series), **logs** (events with context), **traces** (a request's cross-service path), Kubernetes **Events**, and configuration/change history. Each answers a different question. Metrics show rising HTTP 5xx error ratio and p99 latency, logs reveal stack traces, traces show where dependency delays occur, while Events show scheduler/probe/node decisions. Events are not a permanent logging/audit system and can expire quickly. `kubectl logs` often reads limited container logs on nodes; a reliable production system ships them centrally with retention and access control.

The four golden signals—latency, traffic, errors and saturation—are useful for user-facing services. Define an **SLI** as a measured reliability indicator such as good requests / eligible requests; **SLO** is the target over an interval. A 99.9% availability objective over a 30-day month allows roughly 43.2 minutes of budget, depending on exact definition and window. Error budgets let engineers manage release speed versus risk instead of chasing 100% uptime slogans. For resource planning distinguish CPU utilization from saturation, memory RSS/cache, queue depth and tail latency.

A Kubernetes incident triage should preserve evidence. First establish symptom and blast radius. Then check context, workload status, recent deploys, Events, logs, metrics, EndpointSlices, node pressure and dependencies. Avoid restarting everything because it deletes diagnostic state and may increase load. `kubectl logs --previous` helps with crash loops, but previous logs can disappear after many rotations or node failures. Gather process-level evidence (`ss`, `ip`, `journalctl`) only on authorized hosts.

A good incident record gives timestamp, impact, hypotheses, discriminating tests, interventions, monitoring, and a blameless timeline. For senior roles, it is not enough to find 'the pod was OOMKilled': explain why load increased or limits were wrong, how tests would have detected it, and how future blast radius is controlled.

**Figure 17: Monitoring, logs, traces and incident diagnosis**




**How to read this diagram, arrow by arrow:**

- **User request → Service / Ingress** (HTTP request): Real traffic reaches the front door and enters the app routing path.
- **Service / Ingress → App Pod** (route): Eligible backend processes handle requests.
- **App Pod → Metrics + SLI** (emit counters): Counters, histograms, resource telemetry and app SLIs are exported.

- **App Pod → Logs + Events** (record events): App logs and Kubernetes Events report important discrete events.
- **App Pod → Distributed trace** (propagate span): Correlation IDs/spans connect work across dependencies when instrumented.
- **Metrics + SLI → On-call analysis** (dashboard / alert): Metrics quantify symptom scope and duration.
- **Logs + Events → On-call analysis** (error details): Logs and events help explain specific failures.
- **Distributed trace → On-call analysis** (latency path): Traces reveal the path and relative durations of remote operations.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 83: `kubectl get events -n artha-lab --sort-by=.lastTimestamp`

**Where:** HOST
**Why run it:** Inspect recent scheduler/kubelet/controller signals.
**Decode every important part:** Sort Events by their `lastTimestamp` field if present.
**Expected observation:** Chronological event descriptions.
**If the result differs:** Some Events expire or use different timestamps; absence is not proof of health.
**Safety / side effect:** Read only.

```
kubectl get events -n artha-lab --sort-by=.lastTimestamp
```

### Command 84: `kubectl logs deployment/web -n artha-lab --tail=50`

**Where:** HOST
**Why run it:** Read a bounded number of recent application logs.
**Decode every important part:** `logs` targets Deployment-managed Pod, `--tail=50` limits lines.
**Expected observation:** Up to 50 log lines from selected container.
**If the result differs:** If multiple Pods/containers, explicit selection improves confidence.
**Safety / side effect:** Read-only but may contain credentials/user data in real systems.

```
kubectl logs deployment/web -n artha-lab --tail=50
```

### Command 85: `kubectl get pods -n artha-lab -o wide`

**Where:** HOST
**Why run it:** Detect restarts, readiness and placement.
**Decode every important part:** Wide output shows nodes/IP; restart count shows container restarts.
**Expected observation:** A table of state.
**If the result differs:** `Running` alone does not guarantee passing app SLI.
**Safety / side effect:** Read only.

```
kubectl get pods -n artha-lab -o wide
```

### Command 86: `kubectl top pods -n artha-lab`

**Where:** HOST; metrics API exists
**Why run it:** Read present usage, not capacity requests.
**Decode every important part:** `top` uses metrics API; CPU/memory current samples.
**Expected observation:** Pod usage values.
**If the result differs:** No Metrics API is common on minimal kind cluster; skip/record unmet dependency.
**Safety / side effect:** Read only.

```
kubectl top pods -n artha-lab
```

### Command 87: `kubectl describe deployment web -n artha-lab`

**Where:** HOST
**Why run it:** Inspect conditions and deployment-level Events.
**Decode every important part:** Describe summarizes desired/available counts and controller health.
**Expected observation:** Available/Progressing conditions.
**If the result differs:** Need Pods, node and app observations to find root cause.
**Safety / side effect:** Read only.

```
kubectl describe deployment web -n artha-lab
```

## Common faults and diagnostic reasoning

- **Dashboard green while users fail** — Choose SLIs tied to user success, not only CPU/Ready.
- **Restart erases evidence** — Collect Events, previous logs, rollout revisions before interventions.
- **Metrics server absent locally** — `kubectl top` is optional; use available cluster components and state the limitation.

## Guided practice and evidence

1. Write an SLI for successful HTTP requests and compute an example ratio from 999 good / 1000 total.
2. For a synthetic outage, record impact, four hypotheses, tests and evidence.
3. Draft a blameless postmortem with time, trigger, detection, mitigation, contributing factors and action owners.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why can `kubectl get pods` show all Pods Ready while a service SLO is failing?

**Model answer:** Readiness usually checks narrow process health, not every user path or dependency latency. Network failures, wrong routing, database problems and application bugs can still violate error/latency SLIs.

**Official references:** https://sre.google/sre-book/service-level-objectives/, https://kubernetes.io/docs/reference/kubectl/generated/kubectl_logs/

---

# Deep Lesson 18 — Go programming, Kubernetes APIs and client libraries

**Goal:** Build the software engineering prerequisites for controllers and platform services.
**Starting point:** Linux terminal, basic programming concepts and ability to read source code.
**Hinglish mental model:** Kubernetes operator Go mein loop likhta hai: desired aur actual compare karo, idempotent change karo.

## Teach the concept from zero

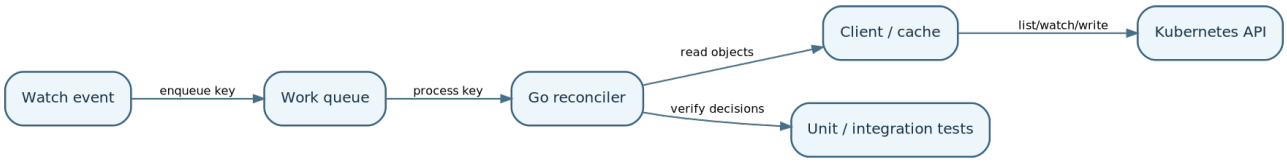
High-compensation Kubernetes engineering often requires writing software, not only managing YAML. **Go** is used extensively in the Kubernetes ecosystem because it compiles to standalone binaries, has simple concurrency primitives, explicit errors and strong libraries. Begin with types, structs, functions, interfaces, pointers and errors, then learn `context.Context`, channels, goroutines and tests. A `context` propagates cancellation/deadlines through API calls; it is not a global configuration bag. A goroutine is a lightweight concurrent execution unit; it does not make shared state race-safe by magic.

Kubernetes client libraries perform API operations using typed objects, dynamic clients or REST calls. **List** returns a snapshot; **watch** streams changes after a resource version and must handle disconnect/relist. A production informer caches resources and reduces unnecessary API load. API request throttling, retries, pagination, authorization and leader election matter at scale. **Optimistic concurrency** via `metadata.resourceVersion` helps prevent clobbering other writers. A 409 Conflict can mean another writer updated the object; re-read and compute a new safe update. Blind tight retry loops cause thundering-herd behavior.

Separate pure business logic from API adapters. Unit-test reconciliation decisions with fake clients when appropriate, but remember fake clients may not enforce admission/defaulting exactly like a real API server. Use integration tests with envtest or disposable clusters for authoritative API behavior. Treat tests as evidence scoped to what they actually covered. A controller should be idempotent (repeated reconciliation converges) and should not treat every event as a one-time order to mutate everything.

On an 8 GB machine, start with a small Go program. The original pack contains a standard-library retry example and tests; use it to practice bounded retries. Later use controller-runtime/Kubebuilder for full operator workflows on a better-provisioned cluster if needed.

**Figure 18: Go programming, Kubernetes APIs and client libraries**




**How to read this diagram, arrow by arrow:**

- **Watch event → Work queue** (enqueue key): Event handlers add reconcile keys to a rate-limited queue, not necessarily copy every object as work.
- **Work queue → Go reconciler** (process key): Reconciler processes one key and is prepared for retries.
- **Go reconciler → Client / cache** (read objects): It reads current state, which may be eventually consistent depending on cache.
- **Client / cache → Kubernetes API** (list/watch/write): Client/informer eventually uses Kubernetes API for state and mutations.

- **Go reconciler → Unit / integration tests** (verify decisions): Pure logic and API interactions should be separately tested to reduce surprises.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 88: `go version`

**Where:** HOST; Go installed
**Why run it:** Confirm compiler/toolchain.
**Decode every important part:** `go version` prints Go toolchain.
**Expected observation:** A `go version go... linux/...` line.
**If the result differs:** Install from trusted package/source only with permission.
**Safety / side effect:** Read only.

```
go version
```

### Command 89: `go test ./...`

**Where:** HOST inside `examples/retry` or Go module
**Why run it:** Run all package tests recursively.
**Decode every important part:** `go test` builds/runs tests; `./...` selects packages under current module.
**Expected observation:** `ok` and timing when tests pass.
**If the result differs:** If not in module, run from correct directory; dependencies may need network unless cached.
**Safety / side effect:** Runs local code; do not test unknown untrusted repositories.

```
go test ./...
```

### Command 90: `go test -race ./...`

**Where:** HOST where race detector is supported
**Why run it:** Check test paths for common data races.
**Decode every important part:** `-race` instruments memory access; `./...` tests all packages.
**Expected observation:** Test success; race detector can use significantly more RAM.
**If the result differs:** Race detector may not be installed/supported or requires CGO/toolchain.
**Safety / side effect:** Local CPU/memory cost; no cloud mutation.

```
go test -race ./...
```

### Command 91: `go vet ./...`

**Where:** HOST inside Go module
**Why run it:** Perform useful static checks.
**Decode every important part:** `go vet` analyzes suspicious code patterns; not a formal proof.
**Expected observation:** No output on clean code.
**If the result differs:** Review warning paths and fix source.
**Safety / side effect:** Local checks only.

```
go vet ./...
```

### Command 92: `kubectl api-resources`

**Where:** HOST
**Why run it:** Discover which API resource kinds exist on actual cluster.
**Decode every important part:** `api-resources` lists groups, names, verbs and namespaced scope.
**Expected observation:** Rows such as deployments.apps and pods.
**If the result differs:** A missing CRD means its resource type is not installed.
**Safety / side effect:** Read only.

```
kubectl api-resources
```

## Common faults and diagnostic reasoning

- **Watch reconnect causes missed event assumption** — Re-list/reconcile current state; do not rely on receiving each intermediate event.
- **Conflict update** — Read resourceVersion, re-fetch and retry with backoff when safe.
- **Fake test passes while real server rejects** — Use integration testing because API defaulting/admission differs from simplistic mocks.

## Guided practice and evidence

1. Read the original retry Go example, identify max attempts and expected return values.
2. Write a function that receives desired/current replicas and returns whether update is necessary; test 0, equal and high values.
3. Explain why an informer cache can be stale and how a controller should respond.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why must an operator reconcile idempotently?

**Model answer:** Watch events can be duplicated, delayed or lost across reconnects. Idempotent reconciliation repeatedly compares current API state to desired state and makes only safe, necessary changes, allowing convergence after failures.

**Official references:** https://go.dev/doc/, https://github.com/kubernetes/client-go, https://book.kubebuilder.io/

---

# Deep Lesson 19 — CRDs, finalizers, reconciliation controllers and operators

**Goal:** Explain how an operator safely extends Kubernetes and write a minimal API design.
**Starting point:** Go basics, Kubernetes API, Deployment owner relationships.
**Hinglish mental model:** CRD naya noun deta hai; controller us noun ka promised behavior implement karta hai.

## Teach the concept from zero

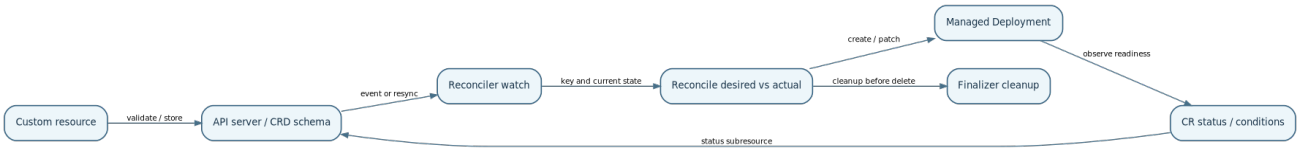
A **CustomResourceDefinition (CRD)** registers a new Kubernetes API kind and its structural schema. It is data/schema, not running automation by itself. A **controller** watches resources and reconciles desired spec against real state; an **operator** typically packages a domain-specific controller that automates operational knowledge for a workload. For example, a `WebApp` custom resource could declare an image and desired replicas. The operator could create/patch a Deployment and expose status conditions about convergence.

A correct API design distinguishes user intent (`spec`) from reported observations (`status`). Make public fields stable, documented, validated and defaulted appropriately. Use **status conditions** with observed generation to describe progress instead of one vague Boolean `ready`. Use **ownerReferences** for dependent namespaced objects so garbage collection can clean up safely; namespace/cluster-scoped ownership rules matter. **Finalizers** let a controller run cleanup logic before an object disappears, but a permanently failing finalizer can block deletion indefinitely. Always make cleanup idempotent and time-bounded with observable errors.

A reconcile function should read the latest object, decide expected owned resources, create/update them only if necessary, report status, and return retry/requeue guidance. Be careful with `resourceVersion` concurrency, patch semantics and event storms. If you create a child Deployment on every reconcile without checking ownership, you will produce duplicate or conflicting resources. If you store sensitive credentials in custom resource status, you have created an exposure.

Operator development exercises should move through unit tests, envtest (real Kubernetes API server semantics without a full cluster in suitable harness), integration tests on kind, conformance tests, security review and canary deployment. Many multi-team platform ideas should first be built with existing controllers (Helm/GitOps/Policy) rather than immediately inventing a CRD. Write one only when there is a stable domain abstraction with a meaningful operational contract.

**Figure 19: CRDs, finalizers, reconciliation controllers and operators**




**How to read this diagram, arrow by arrow:**

- **Custom resource → API server / CRD schema** (validate / store): CRD schema validates the custom resource before it is persisted.
- **API server / CRD schema → Reconciler watch** (event or resync): Controller sees eventual changes through a watch/informer.
- **Reconciler watch → Reconcile desired vs actual** (key and current state): Reconciler gets a key and reads current API state, not blindly obeying an event snapshot.

- **Reconcile desired vs actual → Managed Deployment** (create / patch): It creates or patches dependent workloads to meet user intent.
- **Managed Deployment → CR status / conditions** (observe readiness): Child workload conditions become input to meaningful custom resource status.
- **Reconcile desired vs actual → Finalizer cleanup** (cleanup before delete): On deletion it may perform authorized external cleanup before removing finalizer.
- **CR status / conditions → API server / CRD schema** (status subresource): Controller writes observed status with correct concurrency and permissions.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 93: `kubectl get crds`

**Where:** HOST
**Why run it:** Discover all registered custom resource API types.
**Decode every important part:** `crds` short for CustomResourceDefinitions; cluster-scoped.
**Expected observation:** A list or none, depending on installed addons.
**If the result differs:** CRD names generally end with a DNS-style group name.
**Safety / side effect:** Read only.

```
kubectl get crds
```

### Command 94: `kubectl api-resources --api-group=apps`

**Where:** HOST
**Why run it:** Show registered built-in workload APIs in one group.
**Decode every important part:** `--api-group` filters resource discovery by group.
**Expected observation:** Deployments, ReplicaSets, StatefulSets, etc.
**If the result differs:** If group missing, verify cluster API/version.
**Safety / side effect:** Read only.

```
kubectl api-resources --api-group=apps
```

### Command 95: `kubectl explain deployment.spec.replicas`

**Where:** HOST
**Why run it:** Learn how API discovery supplies field descriptions.
**Decode every important part:** `explain` queries OpenAPI/schema; dotted path selects nested field.
**Expected observation:** Field type and description.
**If the result differs:** May require API access; resource spelling must be correct.
**Safety / side effect:** Read only.

```
kubectl explain deployment.spec.replicas
```

### **Command 96:** `kubectl get deployment web -n artha-lab -o yaml`

**Where:** HOST
**Why run it:** Study observed generation, owner references and status fields.
**Decode every important part:** `-o yaml` prints full object after API defaulting.
**Expected observation:** Metadata, spec and status.
**If the result differs:** Do not commit server-generated fields directly as declarative manifests.
**Safety / side effect:** Read only.

```
kubectl get deployment web -n artha-lab -o yaml
```

## YAML you should be able to write without copying

Save this in a new file in your lab directory, then apply it only after understanding the fields. YAML is indentation-sensitive: use spaces, not tabs.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: webapps.platform.example.org
spec:
  group: platform.example.org
  scope: Namespaced
  names:
    plural: webapps
    singular: webapp
    kind: WebApp
  versions:
  - name: v1alpha1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              replicas: {type: integer, minimum: 0}
            required: [replicas]
```

**Field-by-field explanation:**

- `CustomResourceDefinition`: API extension registration, not a running operator.
- `metadata.name`: plural resource dot API group by convention.
- `spec.group`: fully qualified API group.
- `scope: Namespaced`: CR instances belong to a namespace.
- `names.plural/singular/kind`: API discovery and kind spelling.
- `versions[]`: list of served versions and storage version.
- `v1alpha1`: experimental API that may evolve; production APIs need versioning/conversion plans.
- `openAPIV3Schema`: structural schema for object shape.
- `properties.spec.properties.replicas`: custom user desired replica field; minimum 0.
- `required: [replicas]`: that field must be present for this illustrative CRD.
- Operator/controller Deployment is still separate; CRD alone creates no child workload.

**Change exercise:** Create sample `WebApp` data on paper and write what a separate controller must do to make a Deployment exist.

## Common faults and diagnostic reasoning

- **CRD exists but custom object does nothing** — CRD registers schema only; verify operator Deployment is running and authorized.
- **Finalizer blocks deletion** — Investigate controller health/cleanup dependency; manually removing finalizers can leak resources.
- **Controller loops endlessly** — Check idempotence, defaulting, status-only update loops and rate limits.
- **Security of cross-namespace child objects** — Owner references and RBAC have namespace scope constraints; do not grant cluster-admin by default.

## Guided practice and evidence

1. Design a `WebApp` CRD spec with image/replicas plus status conditions and validation rules.
2. Write controller pseudocode and explain each retry/patch decision.
3. Design a failure test: child Deployment deleted, operator restart, conflicting spec update, CR deletion with external resource cleanup.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** What is a finalizer, and when can it become dangerous?

**Model answer:** A finalizer prevents final object deletion until controller cleanup completes. It is useful for external resource lifecycle but can leave a resource stuck Terminating if cleanup/controller fails; cleanup must be idempotent and observable.

**Official references:**
https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/, https://kubernetes.io/docs/concepts/overview/working-with-objects/finalizers/, https://book.kubebuilder.io/

---

# Deep Lesson 20 — SRE, multi-cluster, security architecture and platform products

**Goal:** Reason about 99.9% SLOs, blast radius, multi-tenancy, fleet upgrades and platform API design.
**Starting point:** You understand cluster HA, observability, GitOps and controllers.
**Hinglish mental model:** Staff engineer ka kaam platform ko usable, secure, reliable aur economical product banana hai.

## Teach the concept from zero

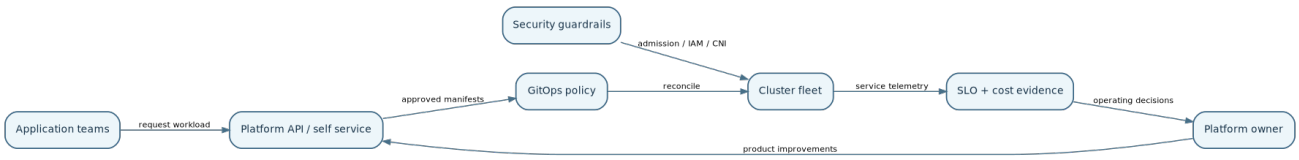
Senior engineers fix many incidents; staff-level platform engineers also change the **system** so incidents are less likely and recovery is fast. A platform is an internal product for developer teams: paved-road templates, guardrails, documentation, self-service provisioning, safe defaults and measured customer adoption. A developer should request an application environment through a small, stable interface while the platform automates policy, identity, scaling and observability behind it. Kubernetes must not be exposed as raw complexity when a smaller internal API will do.

Multi-cluster is useful for workload isolation, regional fault domains, regulatory boundaries, upgrade blast-radius reduction or organizational autonomy—but it adds fleet management, credentials, version skew, application routing, data replication, policy consistency and cost. Two clusters in the same region and same network dependency may not deliver the availability you imagine. **RTO/RPO, dependency mapping and failure-injection evidence** matter more than cluster count. Cross-region applications require explicit data consistency strategies; Kubernetes does not automatically replicate application databases across clusters.

Security architecture spans tenant boundaries: namespaces + RBAC + PSA + NetworkPolicy, quota/limit policies, isolated nodes or clusters for strong trust boundaries, workload identity, image policy and audit. NetworkPolicy isolation depends on CNI enforcement and is not a substitute for hypervisor or kernel isolation. Platform identity should be federated rather than copied as permanent credentials everywhere. Define control ownership: who can create a tenant, change policy, authorize outbound connections, rotate secrets and restore data?

A platform needs product metrics and cost discipline. Track lead time to first deploy, deployment failure rate, team adoption, change rollback time, developer support burden, fleet patch compliance, SLO attainment and total resource cost. Node utilization alone can encourage dangerous overcommit. Use workload-aware capacity models, request rightsizing, purchase options (on-demand, reserved, spot) with interruption analysis, storage retention and egress budgeting. Do not promise a fixed percentage cost reduction without baselines and measured data.

**Figure 20: SRE, multi-cluster, security architecture and platform products**




**How to read this diagram, arrow by arrow:**

- **Application teams → Platform API / self service** (request workload): Developer expresses requirements via stable, tested platform interface.
- **Platform API / self service → GitOps policy** (approved manifests): Platform renders auditable desired state with policy checks and approvals.

- **GitOps policy → Cluster fleet** (reconcile): GitOps controllers manage authorized clusters without exposing every detail to app teams.
- **Security guardrails → Cluster fleet** (admission / IAM / CNI): Security layers enforce tenant and workload rules at relevant boundaries.
- **Cluster fleet → SLO + cost evidence** (service telemetry): Operational metrics and cost feedback reflect real outcomes.
- **SLO + cost evidence → Platform owner** (operating decisions): Owners interpret health, adoption, reliability and expenditure.
- **Platform owner → Platform API / self service** (product improvements): Roadmap decisions improve developer experience and resilience.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 97: `kubectl get namespaces`

**Where:** HOST
**Why run it:** Inspect logical tenant/workload scopes.
**Decode every important part:** Namespaces are cluster-scoped objects; command lists them.
**Expected observation:** Several system and application namespaces.
**If the result differs:** Namespace boundary is not automatically a strong tenant security boundary.
**Safety / side effect:** Read only.

```
kubectl get namespaces
```

### Command 98: `kubectl get resourcequotas -A`

**Where:** HOST
**Why run it:** Review tenant capacity policies across namespaces.
**Decode every important part:** ResourceQuotas are namespaced; `-A` requests all namespaces.
**Expected observation:** Quota object names and namespace placement.
**If the result differs:** Quota presence does not imply correct resource requests or cost controls.
**Safety / side effect:** Read only.

```
kubectl get resourcequotas -A
```

### Command 99: `kubectl get networkpolicies -A`

**Where:** HOST
**Why run it:** Inventory declared network boundaries.
**Decode every important part:** `networkpolicies` are namespaced; show all.
**Expected observation:** Policy inventory.
**If the result differs:** Enforcement depends on CNI; no single command proves complete isolation.
**Safety / side effect:** Read only.

```
kubectl get networkpolicies -A
```

### Command 100: `kubectl get serviceaccounts -A`

**Where:** HOST
**Why run it:** Find workload identities and assess scope.
**Decode every important part:** ServiceAccounts are namespaced API identities.
**Expected observation:** Identity inventory by namespace.
**If the result differs:** Presence does not reveal token mount settings or effective permissions alone.
**Safety / side effect:** Read only.

```
kubectl get serviceaccounts -A
```

### Command 101: `kubectl get pdb -A`

**Where:** HOST
**Why run it:** Find voluntarily protected workloads.
**Decode every important part:** PDB controls certain evictions.
**Expected observation:** PDBs and their namespace scopes.
**If the result differs:** PDB does not protect against all node crashes or forced deletions.
**Safety / side effect:** Read only.

```
kubectl get pdb -A
```

## Common faults and diagnostic reasoning

- **Multi-cluster increases complexity without availability** — Explain which independent failure domain, routing and data replication the second cluster adds.
- **Portal is just a YAML form** — Add tested API contracts, safety guards, docs, support and adoption measures.
- **High utilization reduces reliability** — Model headroom for zone loss, rollout surge, traffic spikes and node replacement.

## Guided practice and evidence

1. Design a platform onboarding flow where a new team gets namespace, identity, policy, quota, dashboard and deployment template.
2. Draw two clusters with separate failure domains; list every shared dependency that can still cause joint outage.
3. Write a 30-day SLO and failure-budget policy with measured indicators and explicit error-budget response rules.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** How would you decide whether one cluster or multiple clusters is appropriate?

**Model answer:** Compare blast radius, trust boundaries, data/regulatory location, latency, resilience targets, operational complexity and cost. Multi-cluster is warranted only when those requirements justify routing, federation, policy and data consistency overhead.

**Official references:** https://sre.google/sre-book/service-level-objectives/, https://kubernetes.io/docs/concepts/security/multi-tenancy/

---

# Deep Lesson 21 — Staff-level system design, interview narratives and the ₹1 crore trajectory

**Goal:** Present end-to-end technical decisions backed by evidence rather than certification lists.
**Starting point:** You have a plan for six portfolio projects and real hands-on lab evidence.
**Hinglish mental model:** High salary kisi ek command ka reward nahi; production impact, judgment, code aur communication dikhani hoti hai.

## Teach the concept from zero

A ₹1 crore annual compensation target is a **career aspiration**, not an outcome that follows automatically from a course. At a small set of employers, total compensation may include base, bonus and annualized equity; guaranteed cash pay may be much lower. Location, experience, role scope, interviews, offer negotiation and equity valuation all matter. Think in capability and evidence milestones, not promises of a numeric salary after exactly 36 weeks.

Build three complementary skill narratives: **deep technical ownership**, **large-system design**, and **cross-team impact**. Deep ownership means you can debug the network path from a user request through DNS, ingress, Service/EndpointSlice, Pod, kubelet, CNI and Linux, and can write small Go tooling/CRDs. System design means you make explicit choices about HA, quorum, RPO/RTO, Kubernetes/Kafka/Postgres failure domains, rollout strategy, security and cost. Cross-team impact means you produce reusable platform interfaces, coach peers, write ADRs, lead incidents calmly and measure developer productivity or reliability improvements.

A staff-level design interview is a conversation. First ask for constraints: traffic distribution, workloads, data requirements, consistency, security, uptime objective, compliance and costs. Draw a minimal baseline architecture. Identify critical failure modes, then propose realistic alternatives and trade-offs. Quantify: QPS, concurrent connections, vCPU/memory requests, disk IOPS, per-zone capacity, backup frequency and recovery window with stated assumptions. Prefer one tested cluster architecture over a fancy but unjustified service mesh plus global multi-cluster mesh.

For a portfolio, produce evidence packs for six major projects: Kubernetes app platform; cluster operations/DR; secure EKS design; GitOps pipeline; Go controller/operator; staff multi-cluster architecture. Each should contain README, diagram, threat model, deployment/test steps, expected outputs, failure injection, runbook, costs, architectural decisions and a clear statement of what was **actually built/tested** versus only designed. Resume bullets should not invent percentages; measured latency, rollback time, incidents prevented and time saved are far stronger than unsupported claims.

A typical progression includes solid senior hands-on capability, senior platform/SRE ownership, then staff/principal-level architectural influence. Cloud certifications may help screening but are neither required nor sufficient. Practice incident deep dives, coding in Go/Python, distributed systems, Terraform, EKS and behavioral leadership stories. Track interviews and the specific competencies you lack; spend the following week's practice on those gaps.

**Figure 21: Staff-level system design, interview narratives and the ₹1 crore trajectory**




**How to read this diagram, arrow by arrow:**

- **Deep fundamentals → Reproducible labs** (test concepts): Concepts are reinforced through verified command/YAML behavior.
- **Reproducible labs → Production-style capstones** (compose systems): Projects combine networking, security, data, IaC, SRE and code.

- **Production-style capstones → Measured business impact** (measure outcomes): Record objective changes where legitimately measured; do not invent numbers.
- **Measured business impact → Staff interview evidence** (explain decisions): Interview panels evaluate experience, architecture, trade-offs and collaboration.
- **Staff interview evidence → Compensation negotiation** (qualify for role): An offer depends on demand, location, company, negotiation and competing candidates—not course completion alone.

## Command-by-command workshop

**Notation:** `HOST` means the Debian terminal running `kubectl`; `POD` means a shell entered in a disposable Pod; `NODE` means an authorized node shell. Commands assume you have checked `kubectl config current-context` and are targeting the local practice cluster. Output examples are illustrative, not measurements from this environment.

### Command 102: `git log --oneline -5`

**Where:** HOST inside your portfolio Git repository
**Why run it:** Review last five commits that demonstrate sustained work.
**Decode every important part:** `git log` reads history; `--oneline` makes compact entries; `-5` limits to five.
**Expected observation:** Commit hashes and descriptions.
**If the result differs:** If no commits, do the work before claiming experience.
**Safety / side effect:** Read only.

```
git log --oneline -5
```

### Command 103: `git status --short`

**Where:** HOST in portfolio repo
**Why run it:** Prevent presenting uncommitted artifacts as released work.
**Decode every important part:** Shows modified/untracked files.
**Expected observation:** Clean repo means no uncommitted changes.
**If the result differs:** Check ignored files too if documenting evidence completeness.
**Safety / side effect:** Read only.

```
git status --short
```

### Command 104: `kubectl config current-context`

**Where:** HOST
**Why run it:** Verify cluster identity in any recorded demo.
**Decode every important part:** Shows kubeconfig context; safe preflight.
**Expected observation:** The expected disposable practice cluster.
**If the result differs:** If context differs stop before mutation.
**Safety / side effect:** Read only.

```
kubectl config current-context
```

### Command 105: `kubectl get deployments -A -o wide`

**Where:** HOST
**Why run it:** Capture honest scope of application deployments.
**Decode every important part:** `-A` spans namespaces; `-o wide` adds extra fields.
**Expected observation:** Actual visible objects, which may be none in fresh cluster.
**If the result differs:** Do not infer availability or impact only from a snapshot.
**Safety / side effect:** Read only.

```
kubectl get deployments -A -o wide
```

## Common faults and diagnostic reasoning

- **Only memorized YAML** — Practise describing control loops and diagnosing failure modes from actual evidence.
- **Portfolio claims not reproducible** — Include scripts, inputs, sanitized outputs, expected vs actual and runbooks.
- **Compensation target mistaken for guarantee** — Distinguish base, cash, bonus and annualized equity with vesting/valuation risks.

## Guided practice and evidence

1. Record a 15-minute answer to "Design a private, multi-tenant platform serving 100 teams" and assess assumptions, architecture, failure modes, security, costs and migration.
2. Write six STAR stories based on actual work: incident, outage prevention, automation, security change, migration and collaboration.
3. Create a salary comparison spreadsheet distinguishing fixed salary, joining bonus, recurring bonus and annualized equity from actual offer documents.

Do not mark this lesson complete merely because the command exited with status 0. Save the **before/after**, describe why the system changed, and clean up disposable resources when appropriate.

## Interview rehearsal

**Question:** Why should a staff engineer know Kubernetes controllers and Go if managed EKS exists?

**Model answer:** Managed control planes reduce infrastructure toil, but platform problems remain: policy integration, secure abstractions, developer APIs, reliability tooling, scale-aware controllers and incident automation. Writing trustworthy software is part of senior platform value.

**Official references:** https://kubernetes.io/docs/concepts/architecture/controller/, https://sre.google/sre-book/

---